

Leakage-Controlled Concept Graph Construction and Cold-Start Diagnostic Protocol for Knowledge Tracing

Dao Minh Tuan ¹, Nguyen Khanh Trinh ¹,
Nguyen Tien Duong ¹, Ngo Quoc Khanh ³,
Nguyen Van Hau ^{1*}, Le Hoang Son ²

^{1*}Department of Quality Assurance and Testing, Hung Yen University
of Technology and Education, Hung Yen, Vietnam.

²Information Technology Institute, Vietnam National University, 144
Xuan Thuy Street, Cau Giay, Hanoi, Vietnam.

³Advanced International Research Center on Applied Artificial
Intelligence, Information Technology Institute, Vietnam National
University, Hanoi, Vietnam.

*Corresponding author(s). E-mail(s): [nvhou666@gmail.com](mailto:nvhau666@gmail.com);
Contributing authors: tuanymc@utehy.edu.vn; trinhnk@utehy.edu.vn;
duongnt@utehy.edu.vn; quockhanhngo.official@gmail.com;
sonlh@vnu.edu.vn;

Abstract

Graph-enhanced knowledge tracing (KT) often infers concept graphs from the same interaction logs used for training and evaluation. When edges are pooled before the train/test split, held-out information can reach the model through the graph even if the KT optimiser sees only the training fold. We present a leakage-controlled *audit protocol*—not a new KT backbone—for knowledge-component (KC) graph construction: learner-based temporal splits, train-only prerequisite inference (optional similarity where the Q-matrix supports it), DAG validation with cycle pruning, KC-level cold-start stratification, a practitioner checklist, and a DAG Disruption Rate (DDR) diagnostic. We instantiate the protocol on XES3G5M (primary model-comparison benchmark), ASSISTments 2012 (secondary), Junyi Academy (saturated sanity + ground-truth cross-validation), and two synthetic logs. The audit yields acyclic train-only prerequisite graphs on all corpora; the DDR sweep separates five augmentation operators, with a

prerequisite-preserving operator measurably less disruptive than edge dropping at a matched budget. An *exploratory* DGEKT-only downstream stress test on ASSISTments and XES3G5M finds near-zero AUC changes despite large structural perturbation; we do not extrapolate that null sensitivity to other backbones. On XES3G5M the audit surfaces a *model-specific* ordering within the **primary** comparison trio (*simpleKT*, stock pyKT GKT, native GIKT): under the released 10-epoch training budget (Table S15), GKT trails *simpleKT* by ≈ 0.041 AUC (paired-*t* 95% CI $[-0.044, -0.038]$; Table S16). A GKT *epoch-extended* ablation (30 epochs, batch 32; Table S21–S22; *simpleKT* reference unchanged at 10 epochs, batch 64) recovers $\approx +0.003$ AUC on seed **42** but leaves pooled mean $\Delta \approx -0.038$ (95% CI $[-0.040, -0.035]$ over nine folds across split seeds **17**, **42**, and **1234**): the GKT under-performer ordering on XES3G5M persists under this partially matched ablation. Ground-truth cross-validation on Junyi shows node-level agreement but edge-level divergence between behavioural inference and expert curation. A deliberate full-log graph variant shifts headline metrics by at most **0.003** on the three public benchmarks studied here; we position the protocol as reproducibility infrastructure and a practitioner checklist for graph-provenance decisions, not as a route to higher headline accuracy.

Keywords: Knowledge tracing, Concept graphs, Leakage control, DAG audit, Ground-truth cross-validation, Cold-start diagnostics, Reproducibility

1 Introduction

Knowledge tracing (KT) estimates a learner’s latent mastery of knowledge components from interaction history, supporting adaptive practice, intelligent tutoring, and curriculum recommendation. The field has progressed from Bayesian Knowledge Tracing Corbett and Anderson (1994) to deep sequential models Piech et al. (2015); Zhang et al. (2017); Ghosh et al. (2020); Liu et al. (2023a); Pandey and Karypis (2019); Shen et al. (2020) and graph-enhanced models that propagate information over relations between knowledge components Nakagawa et al. (2019); Tong et al. (2020); Yang et al. (2021); Cheng et al. (2024); Cui et al. (2024); Tong et al. (2022). These graph-based models rely on auxiliary concept graphs, usually encoding prerequisite or similarity relations.

We use *data leakage* to mean any situation in which information that would be unavailable at deployment time influences training or evaluation—for example, when test-fold interactions support edges in an auxiliary concept graph even though the KT optimiser trains only on the training fold Kaufman et al. (2012); Kapoor and Narayanan (2023). We use *cold-start* in a KT-specific sense at the knowledge component level: a KC is cold-start when it has too few train-fold interactions to support stable parameter or neighbourhood estimates, in contrast to user- or item-level cold-start in recommender systems Schein et al. (2002).

Current practice leaves three reproducibility gaps that motivate our audit. First, concept graphs are often inferred from the same logs used to train and evaluate

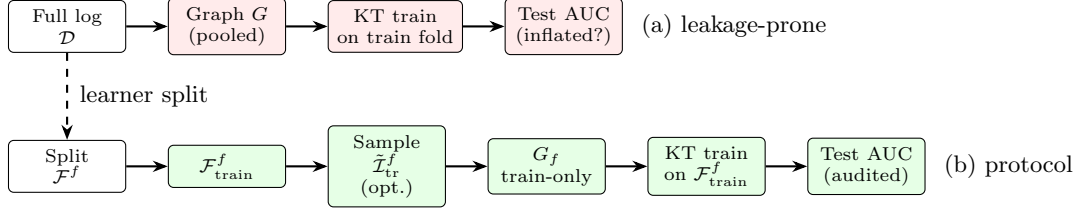


Fig. 1: Illustrative comparison of graph construction workflows. **(a)** Pooling the full log before edge inference lets held-out transitions influence G even when the KT optimiser sees only the train fold. **(b)** Our protocol builds a fold-specific graph G_f from train-only interactions. When train folds are large, edge inference may use a representative train-only sample $\tilde{\mathcal{I}}_{\text{tr}}^f \subseteq \mathcal{F}_{\text{train}}^f$ (optional; Section 3.4) to bound compute while preserving leakage control; reported benchmarks use the full train fold.

KT models; pooling the full log before learner-based splitting lets held-out transitions enter the graph channel even when sequence training is fold-clean Nakagawa et al. (2019); Tong et al. (2020); Liu et al. (2022). The broader ML literature documents leakage as a driver of irreproducible claims Kaufman et al. (2012); Kapoor and Narayanan (2023); within KT, pyKT and follow-on work address *sequence-level* label leakage Liu et al. (2022); Badran and Preisach (2024), but fold-specific audit of *graph provenance*—including bounded train-only sampling when logs are very large—remains under-specified. Second, generic graph augmentations designed for undirected graphs can disturb directional prerequisite structure without measurement You et al. (2020); Zhu et al. (2021). Third, “cold-start” is used inconsistently in KT reporting, so aggregate AUC can mask tail-knowledge component behaviour.

Two short examples make the audit concrete. (1) **Graph-mediated leakage:** if an edge $c_i \rightarrow c_j$ is retained because many *test-fold* learners traverse c_i then c_j , the graph can encode future transitions while the optimiser never sees those learners. (2) **KC cold-start:** if a KC appears only a handful of times in $\mathcal{F}_{\text{train}}^f$ but frequently in $\mathcal{F}_{\text{test}}^f$, graph propagation may rely on unstable neighbourhood evidence that aggregate metrics hide.

Concretely, let $\mathcal{D} = \{(u, t, q, c, y)\}$ be a KT interaction log with learner u , timestamp t , item q , tagged knowledge component c , and correctness label $y \in \{0, 1\}$. A learner-based temporal split $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ assigns each learner to exactly one fold (we use ratios (0.7, 0.1, 0.2) in all reported experiments). For evaluation fold f , the protocol must construct a fold-specific concept graph $G_f = (\mathcal{C}, E_{\text{pre}}^f \cup E_{\text{sim}}^f)$ using *only* $\mathcal{F}_{\text{train}}^f$, or a representative train-only sample $\tilde{\mathcal{I}}_{\text{tr}}^f \subseteq \mathcal{F}_{\text{train}}^f$ drawn entirely within that fold when logs exceed a compute budget (Section 3.4), then train a KT model on $\mathcal{F}_{\text{train}}^f$ and score predictions on $\mathcal{F}_{\text{test}}^f$. The central question is whether reported graph-augmented KT gains remain valid when G_f is audited for leakage, acyclicity, and cold-start coverage.

Figure 1 contrasts the two workflows. In workflow (a), prerequisite or similarity edges may be supported by validation/test interactions; the KT module then appears to generalise while part of the structural input already encodes future behaviour.

Workflow (b) closes this channel by making graph construction a first-class, fold-conditioned step.

Leakage-controlled concept graphs matter wherever KT systems drive scheduling and gating decisions in ITS platforms, MOOC pipelines, and corporate training [Heffernan and Heffernan \(2014\)](#); [Junyi Academy \(2015\)](#); [Choi et al. \(2020\)](#); [Liu et al. \(2023b\)](#). Without fold-specific audits, reported AUC improvements are difficult to attribute to modelling versus preprocessing. For *Applied Intelligence* readers, the deliverable is an audit layer that improves *trustworthy deployment*: provenance artefacts, DAG validity checks, cold-start strata, and tiered diagnostics that help practitioners decide *whether and how* to deploy graph-augmented KT, even when headline accuracy does not move. On our primary benchmark XES3G5M, leakage-controlled evaluation separates graph backbones more clearly than on repetition-heavy corpora: under train-only E_{pre} and three-fold learner CV, the **primary** backbone trio is *simpleKT* (≈ 0.875), stock pyKT GKT (≈ 0.834 ; mean $\Delta \approx -0.041$, 95% CI $[-0.044, -0.038]$ in Table S16; Tables S21–S22 report GKT epoch-extended training (≈ 0.837 on seed 42; pooled $\Delta \approx -0.038$, 95% CI $[-0.040, -0.035]$; *simpleKT* reference held at 10 epochs), and native GIKT (≈ 0.878). The gap is *observational* under fixed epoch caps (30 for sequence checkpoints vs. 10 for graph backbones; Table S15), not a causal attribution of graph failure on XES3G5M under this protocol. SKT and DyGKT rows in Tables 7 and 8 are fork wiring sanity checks only (Supplementary Tables S1–S5 for ACC/NLL) and are excluded from primary ranking claims. A team that deploys from leaky full-log graphs (the default in original GKT/SKT pipelines [Nakagawa et al. \(2019\)](#); [Tong et al. \(2020\)](#)) without provenance logs, similarity-edge feasibility checks, or cold-start strata may reach different *reporting and risk* conclusions than one that runs this protocol first—even when train-only versus full-log ablation moves headline AUC by at most 0.003 (Section 4.5).

We address the gaps above at the level of protocol rather than backbone design. We do not propose a new KT architecture. Instead, we release a leakage-controlled graph construction and audit protocol, a structural augmentation diagnostic (DDR), ground-truth cross-validation (instantiated on Junyi Academy [Junyi Academy \(2015\)](#); [Chang et al. \(2015\)](#)), and cold-start KC reporting, evaluated on ASSISTments 2012 and XES3G5M [Liu et al. \(2023b\)](#) alongside two synthetic sanity logs. Our novelty is a reproducible *audit layer* around concept-graph construction—not another sequence model [Abdelrahman et al. \(2023\)](#); [Liu et al. \(2022\)](#)—that separates structural disruption from downstream AUC, which graph-KT and contrastive-KT papers typically omit [Nakagawa et al. \(2019\)](#); [Lee et al. \(2022\)](#).

Each contribution below answers a specific limitation (L) observed in current graph-KT practice; residual risks are stated explicitly rather than deferred to the Discussion alone.

- **C1 (addresses L1: graph leakage).** A train-only protocol for constructing and auditing KC graphs with prerequisite edges and optional similarity edges $G = (\mathcal{C}, E_{\text{pre}} \cup E_{\text{sim}})$ for KT, including split checks, edge provenance, per-relation availability reporting, and DAG validation with cycle pruning. *Residual risk*: the protocol audits declared graph provenance and tiered diagnostics; it does not certify every possible leakage path in proprietary feature pipelines.

- **C2 (addresses L2: augmentation disrupts DAGs).** The DDR metric, which quantifies how generic graph augmentations disrupt prerequisite assumptions on educational DAGs, instantiated with four formally defined label-agnostic operators and one prerequisite-preserving operator; the latter is measurably less disruptive at a matched perturbation budget, showing that DDR discriminates augmentation operators at the structural level rather than restating edge counts alone. *Residual risk:* downstream predictive sensitivity was stress-tested only on DGEKT (Section 4.7); we do not claim DDR predicts accuracy for other graph consumers.
- **C3 (addresses L3: cold-start reporting).** A cold-start KC diagnostic with explicit frequency-based strata and reproducible scripts on three KT benchmarks. *Residual risk:* stratum thresholds are fixed; very-cold counts can be small on real corpora and should not be over-interpreted.
- **C4 (addresses L4: expert vs. inferred graphs).** A ground-truth cross-validation procedure that quantifies, on Junyi Academy, how train-only behavioural inference of E_{pre} agrees with the expert prerequisite annotation of Chang et al. Junyi Academy (2015), using edge-level, direction-level, and reachability-level metrics across a sweep of top- K cutoffs. *Residual risk:* GT cross-validation is Junyi-only at a single expert threshold ($\theta=0.5$).
- **C5 (addresses L5: benchmarking under audit).** An anchored empirical observation on XES3G5M that backbone ordering under leakage-controlled graphs is *model-specific* within the primary trio (*simpleKT*, pyKT GKT, native GIKT): GKT trails *simpleKT* by mean $\Delta\text{AUC} \approx -0.041$ (95% CI $[-0.044, -0.038]$; Table S16) under the released 10-epoch budget, while GIKT remains competitive. A GKT epoch-extended rerun (30 epochs, batch 32; Table S21–S22; reference *simpleKT* at 10 epochs, batch 64) recovers $\approx +0.003$ mean AUC on seed 42 but leaves pooled $\Delta \approx -0.038$ (95% CI $[-0.040, -0.035]$; nine folds, seeds 17/42/1234): the trio ordering on XES3G5M is *partially epoch-sensitive* yet invariant in direction (GKT trails). Train-only versus full-log ablation ($|\Delta\text{AUC}| \leq 0.003$ on the three public benchmarks) bounds how much graph provenance alone moves headline metrics here, so C5 is a *benchmarking boundary result*, not evidence that leakage control systematically flips deployment rankings. *Residual risk:* three-fold CV limits inferential power per seed; native SKT/DyGKT runs are wiring diagnostics only.

We distill five reporting items for graph-augmented KT papers in Section 5.2 (fold provenance, cold-start strata, matched sequence baselines, ground-truth cross-validation when available, and structure-preserving versus damaging perturbation sensitivity). Supplementary Table S11 reports fold-level tests for transparency; fold-level ΔAUC magnitudes in Table 8—an order larger than fold-to-fold variation—together with Table S16 intervals characterise the primary XES3G5M trio under audit, with Tables S21–S22 isolating the epoch-budget confound across three split seeds with fold-aligned graph rebuilds.

We assign explicit roles to the public corpora studied here. XES3G5M is **primary** (low KC-repeat, non-saturated AUC, clearest graph-backbone separation); ASSISTments 2012 is **secondary** (leakage ablation and cold-start); Junyi Academy is a **saturated sanity** corpus (ground-truth cross-validation and DAG density, not

headline model ranking). Synthetic C2/C5 exercise tiny-graph audit modules. Model-comparison claims in C5 and Section 5.1 are anchored on XES3G5M unless noted. Table 7 reports cross-dataset diagnostic AUC; per-dataset ACC/NLL tables with 95% bootstrap confidence intervals are in Supplementary Tables S1–S5. All baselines use train-only exported graphs and valid+test sequence positions under three-fold learner CV. Protocol metrics (DDR, leakage tables, DAG audit) use the same split configuration; DAG audit and ground-truth cross-validation prose refer to fold 0 unless noted. Similarity edges are empty on ASSISTments (single-skill Q-matrix) and sparse on XES3G5M; Junyi is the only corpus where both E_{pre} and E_{sim} are populated under the default rule. On the three public benchmarks studied here, controlling the graph-leakage channel does not by itself yield large accuracy corrections (Section 4.5): the train-only versus full-log ablation moves headline metrics by at most 0.003 AUC/ACC. We therefore present leakage control as an applied reproducibility and decision-support contribution together with a measured boundary result, not as a route to higher headline numbers, and we do not claim that the protocol eliminates every possible form of leakage or make claims about real-world learning outcomes.

2 Related Work

KT progressed from BKT Corbett and Anderson (1994) to deep sequence models (DKT, AKT, *simpleKT*) Piech et al. (2015); Ghosh et al. (2020); Liu et al. (2023a) and libraries such as pyKT Liu et al. (2022); Abdelrahman et al. (2023). Within KT, leakage work has centred on *label leakage* in expanded KC sequences Liu et al. (2022); Badran and Preisach (2024); the broader ML literature catalogues additional leakage types Kaufman et al. (2012); Kapoor and Narayanan (2023). The complementary channel we study—held-out information entering through the *auxiliary concept graph*—is not fixed by sequence-level remedies and remains under-audited in KT benchmarks.

Positioning against reproducibility and graph-audit practice.

KT benchmark libraries Liu et al. (2022); Abdelrahman et al. (2023) and label-leakage audits Badran and Preisach (2024) standardise sequence splits and expanded-label hygiene; broader reproducibility work catalogues leakage modes beyond a single train/test divide Kaufman et al. (2012); Kapoor and Narayanan (2023). Our incremental contribution is not another split routine but a *fold-conditioned graph provenance layer*: exported edge lists, DAG repair logs, tiered scalar diagnostics, and practitioner reporting items that sequence-level KT fixes do not supply. Concretely, relative to pyKT Liu et al. (2022) split discipline alone, the audit layer adds: (i) fold-specific exported edge artefacts (`e\pre\train\only.csv`, `e\sim\train\only.csv`) with per-edge provenance; (ii) logged DAG cycle pruning rather than silent repair; (iii) tiered leakage functionals plus a controlled injection stress test (Section 4.3) that shows $\text{ECR}_f^{\text{flag}}$ can stay zero while builder-level mass shifts; (iv) DDR structural diagnostics decoupled from AUC; (v) ground-truth cross-validation where expert graphs exist; and (vi) KC cold-start strata tied to train-fold counts. We complement—rather than replace—pyKT-style label audits by auditing the auxiliary graph channel those audits omit.

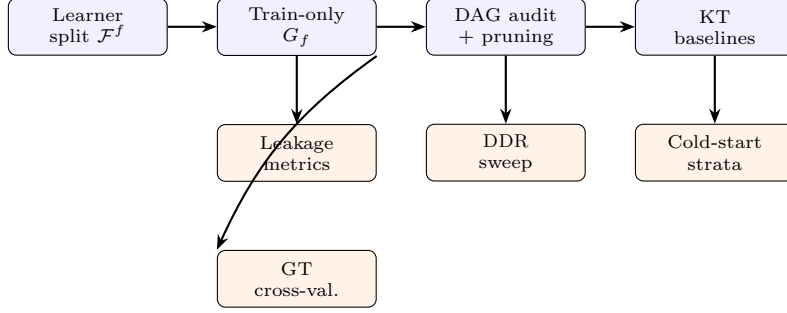


Fig. 2: Research framework connecting protocol modules (Section 3) and experiment tracks (Section 4). Fold-conditioned provenance, DAG repair, structural diagnostics decoupled from AUC, and optional expert cross-validation are the core audit outputs.

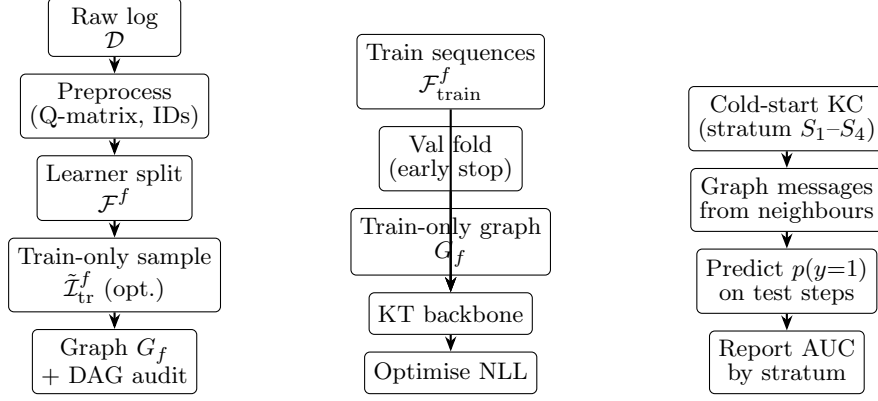
Graph-KT models (GKT, SKT, GIKT, DyGKT, DGEKT) Nakagawa et al. (2019); Tong et al. (2020); Yang et al. (2021); Cheng et al. (2024); Cui et al. (2024) propagate mastery over KC graphs, while prerequisite-learning work infers directed relations from logs, content, or experts Pan et al. (2017); Bai et al. (2025); Chang et al. (2015). Graph-KT papers typically treat the graph as given and optimise predictive loss. Original GKT and SKT pipelines pool transition statistics over the full log before learner splitting Nakagawa et al. (2019); Tong et al. (2020); our protocol isolates that provenance choice. Expert annotations (e.g. Junyi) and behavioural inference capture related but non-identical structure; we quantify disagreement rather than treating one as a proxy for the other.

Graph SSL and contrastive KT import label-agnostic augmentations You et al. (2020); Lee et al. (2022); Kipf and Welling (2017), but educational prerequisite DAGs impose directionality that standard drops can violate. KT reproducibility has focused on code and splits Liu et al. (2022) rather than graph audit logs. Our work is orthogonal to new KT architectures: it supplies an audit layer—leakage diagnostics, DAG pruning logs, DDR, ground-truth cross-validation, and cold-start strata—that graph-KT and contrastive-KT papers typically omit.

3 Protocol

Let $\mathcal{D}$ be a KT interaction log, $\mathcal{C}$ the set of knowledge components, and $\mathcal{F} = (\mathcal{F}_{\text{train}}, \mathcal{F}_{\text{val}}, \mathcal{F}_{\text{test}})$ a learner-based temporal split. The protocol constructs a fold-specific graph $G_f = (\mathcal{C}, E_{\text{pre}}^f \cup E_{\text{sim}}^f)$ using only $\mathcal{F}_{\text{train}}^f$. Figures 2 and 3 summarise module dependencies and artefact-level data flow; Figure 4 details prerequisite-edge construction.

Figure 2 summarises module dependencies. Split discipline is a hard precondition: without learner-disjoint folds, downstream diagnostics are not interpretable. Graph construction and DAG audit are coupled: train-only inference can introduce cycles, so pruning must be logged before any graph-augmented KT run. Leakage metrics, DDR, cold-start strata, and ground-truth cross-validation are *parallel* audits—they do not



(a) Data pipeline from raw log (b) Training inputs and validation checkpoint selection. (c) Cold-start inference and stratum reporting.

Fig. 3: End-to-end protocol plumbing: (a) data preparation and fold-conditioned graph construction, (b) KT training with graph prior and validation early stopping, and (c) how cold-start knowledge components are evaluated and reported separately from aggregate AUC (Section 4.8).

retrain models—which keeps the protocol lightweight but means it cannot certify every possible leakage path (e.g. hidden features in proprietary pipelines).

Figure 3 complements the module map of Figure 2 with artefact-level data flow. Panel (a) shows where optional within-train sampling sits relative to preprocessing and DAG audit; panel (b) shows that graph and sequence inputs are consumed jointly during optimisation; panel (c) shows that test rows inherit KC strata defined from train-fold counts only.

3.1 Formal Problem Statement and Leakage Audit

For each evaluation fold f , the protocol seeks a graph $G_f = (\mathcal{C}, \mathcal{E}_f)$ with $\mathcal{E}_f = E_{\text{pre}}^f \cup E_{\text{sim}}^f$ that is *feasible* with respect to train-only evidence and structural constraints, then verifies low direct leakage on scalar diagnostics. Let $\mathcal{I}_{\text{tr}}^f$ denote train-fold interactions and $\mathcal{H}_f = \mathcal{F}_{\text{val}}^f \cup \mathcal{F}_{\text{test}}^f$ held-out interactions.

Decision variables.

Edge sets $E_{\text{pre}}^f \subseteq \mathcal{C} \times \mathcal{C}$ and $E_{\text{sim}}^f \subseteq \mathcal{C} \times \mathcal{C}$, edge weights $w_{ij} \geq 0$, and pruning decisions on cyclic subsets of E_{pre}^f .

Design intent (not a deployed solver).

Eq. (1) states a *conceptual* objective—maximise retained prerequisite and similarity mass subject to sparsity and acyclicity—but the released code does *not* solve it as a joint numerical programme. Prerequisite and similarity edges are inferred in separate

greedy pipelines with $\lambda=0$ in practice; sparsity enters through support quantiles, top- k per-node caps, and a global edge budget K (Section 3.4), followed by deterministic cycle pruning. Treat Eq. (1) as design documentation for constraints (C3)–(C4), not as an optimiser that is executed end-to-end.

$$\max_{E_{\text{pre}}^f, E_{\text{sim}}^f} \sum_{(i,j) \in E_{\text{pre}}^f} w_{ij} + \lambda \sum_{(i,j) \in E_{\text{sim}}^f} \text{sim}(c_i, c_j) \quad (1)$$

where sim is Jaccard or PMI (Section 3.4) and $\lambda \geq 0$ would trade prerequisite mass against similarity mass if the two pipelines were merged (they are not in the released implementation).

Constraints.

$$(C1) \forall e \in \mathcal{E}_f, S_{\text{hi}}(e) = \emptyset; \quad (2)$$

$$(C2) \mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{va}} = \emptyset, \\ \mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{te}} = \emptyset, \\ \mathcal{U}_f^{\text{va}} \cap \mathcal{U}_f^{\text{te}} = \emptyset; \quad (3)$$

$$(C3) E_{\text{pre}}^f \text{ is acyclic after pruning}; \quad (4)$$

$$(C4) |E_{\text{pre}}^f| \leq K, |\text{Out}_{E_{\text{pre}}^f}(i)| \leq k; \\ \text{sim}(c_i, c_j) \geq \tau \forall (i, j) \in E_{\text{sim}}^f. \quad (5)$$

C1–C2 block cross-fold and cross-learner contamination; C3–C4 encode the DAG audit and top- K /top- k filters of Section 3.4. We deliberately do *not* impose $\text{TBMR}_f=0$ (Eq. (9)) as a feasibility constraint: temporal-transition inference necessarily draws evidence from each learner’s full train timeline, so a zero rate would be attainable only by discarding late-train evidence. TBMR_f is instead reported as a within-train stress diagnostic in the sanity/stress tier below; the released runs accordingly show non-zero values on dense corpora (Table 5).

Audit functionals (diagnostics, not training losses).

Because C1 cannot always be verified from AUC alone, the protocol reports four scalar functionals on G_f : $\text{ECR}_f^{\text{flag}}$ (Eq. (6)), $\text{ECR}_f^{\text{overlap}}$ (Eq. (7)), $|\rho_f|$ (Eq. (8)), and TBMR_f (Eq. (9)). We treat them in three tiers: **primary pass/fail**—fold-specific provenance logs, $\text{ECR}_f^{\text{flag}}=0$, and (when available) ground-truth cross-validation; **empirical bounds**—train-only versus full-log $\Delta\text{AUC}/\text{ACC}$ and cold-start strata; **sanity/stress**— $\text{ECR}_f^{\text{overlap}}$, $|\rho_f|$, and TBMR_f . Under learner-disjoint splits, $\text{ECR}_f^{\text{flag}}=0$ is expected and does not by itself certify graph quality; near-unity overlap can occur when curricula repeat across disjoint learners. TBMR_f quantifies *within-train* temporal mixing (Section 3.2), not cross-split leakage. High AUC does not certify absence of leakage; the tiered diagnostics complement discrimination scores.

Table 1: Taxonomy of leakage classes audited by the protocol: typical source of contamination, a concrete KT example, and the primary direct diagnostic used in this paper (Section 3.2).

Class	Source	Example	Metric
Structural	Learner identifiers reused across train, validation, and test folds	The same learner contributes interactions to both train and held-out partitions under fold f	ECR^{flag}
Edge	Interactions outside $\mathcal{F}_{\text{train}}^f$ used as evidence for edges	Prerequisite or similarity edge retained because val/test transitions inflate co-occurrence or transition counts	$\text{ECR}^{\text{overlap}}$
Target	Outcomes or mappings pooled across splits	Q-matrix or calibration statistics fit on all learners; edge weights implicitly tuned against aggregated correctness	$ \rho $
Temporal	Violating the timestamp boundary τ_u per learner u	Post-boundary attempts included when counting $c_t \rightarrow c_{t+1}$ support or when building time-sensitive features	TBMR
Cold-start neighbourhood	Tail-knowledge component edges dominated by non-train support	Rare-knowledge component similarity edge driven mainly by interactions that belong to val/test under the split	$\text{ECR}^{\text{overlap}} (+ \text{strata})$

3.2 Leakage Control

As introduced in Section 1, leakage here means any information path by which held-out labels, held-out interactions, future timestamps, learner identities, or graph evidence influence graph construction, feature construction, model selection, or evaluation although such information would not be legitimately available at the corresponding prediction point. Section 1 gives the intuitive definition; here we specialise it to *edge provenance* in KC graphs (Table 1), following the learn–predict separation view in data mining Kaufman et al. (2012); Kapoor and Narayanan (2023) and complementing sequence-level label leakage in KT Liu et al. (2022); Badran and Preisach (2024).

We audit five leakage classes at the level of edge provenance (Table 1): structural, edge, target, temporal, and cold-start-neighbourhood classes cover learner pooling, held-out transition support, outcome-conditioned graph artefacts, boundary violations, and tail-knowledge component edges dominated by non-train mass.

Each inferred edge is logged with its fold of origin, timestamp range, relation type, and train-only flag; severe violations halt the pipeline. In addition, we report four scalar *direct* leakage diagnostics that do not reduce to predictive accuracy. Let $\mathcal{U}_f^{\text{tr}}$, $\mathcal{U}_f^{\text{va}}$, and $\mathcal{U}_f^{\text{te}}$ denote the learner-id sets in train, validation, and test under fold f . The learner overlap flag is

$$\text{ECR}_f^{\text{flag}} := \mathbb{K}[(\mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{va}}) \cup (\mathcal{U}_f^{\text{tr}} \cap \mathcal{U}_f^{\text{te}}) \cup (\mathcal{U}_f^{\text{va}} \cap \mathcal{U}_f^{\text{te}}) \neq \emptyset]. \quad (6)$$

When folds are learner-disjoint, $\text{ECR}_f^{\text{flag}} = 0$ for every fold—the regime enforced below.

Let $\mathcal{E}_f$ denote the graph edge set (prerequisite and similarity) consumed when evaluating fold f , and let $\mathcal{H}_f = \mathcal{F}_{\text{val}}^f \cup \mathcal{F}_{\text{test}}^f$ denote held-out interactions under that split. For edge e , let $S_{\text{hi}}(e)$ be held-out interaction tokens counted toward e ’s evidence

and $\chi_{\text{ECR}_{\text{overlap}}}(e) = \mathbb{I}[|S_{\text{hi}}(e)| > 0]$. The held-out pattern overlap rate is

$$\text{ECR}_f^{\text{overlap}} = \frac{1}{|\mathcal{E}_f|} \sum_{e \in \mathcal{E}_f} \chi_{\text{ECR}_{\text{overlap}}}(e). \quad (7)$$

For Edge–Outcome Correlation, stack normalized edge weights into $\mathbf{w}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ (for ties, use the order exported by the audit log). Let $\mathbf{y}_f \in \mathbb{R}^{|\mathcal{E}_f|}$ carry test-fold outcome summaries aligned to each edge—for example, endpoint averages of correctness on $\mathcal{F}_{\text{test}}^f$ or their logits—computed *after* graph construction (labels are used only for this diagnostic, not for inferring $\mathcal{E}_f$). The primary edge–outcome diagnostic is the Pearson correlation

$$\rho_f = \text{Corr}(\mathbf{w}_f, \mathbf{y}_f), \quad (8)$$

computed over edges with defined variance; $|\rho_f| \approx 0$ indicates that retained edge mass is statistically unrelated to test-fold outcome summaries.

Let $t_{u,i}$ be the timestamp of interaction i for learner u and let τ_u be u ’s temporal split cutoff for fold f . Write $\mathcal{I}_{u,f}^> = \{i : t_{u,i} > \tau_u\}$ for post-boundary interactions and $\mathcal{V}_{u,f}$ for the subset of those tokens that incorrectly appear in any multiset counted toward $\mathcal{E}_f$ or in graph-derived features at inference for fold f . The within-train Temporal Boundary Mixing Rate is

$$\text{TBMR}_f = \frac{\sum_u |\mathcal{V}_{u,f}|}{\sum_u |\mathcal{I}_{u,f}^>|}. \quad (9)$$

We instantiate τ_u within $\mathcal{F}_{\text{train}}^f$ at the 70% chronological split of each learner’s train sequence. TBMR_f is therefore a *within-train temporal stress statistic*: it measures how much edge evidence is supported by the late tail of each learner’s *training* sequence after an internal chronological cutoff. It must not be read as cross-fold leakage ($\text{ECR}_f^{\text{flag}}=0$ already certifies learner-disjoint splits). Elevated TBMR_f on dense multi-skill corpora (e.g. XES3G5M) flags reliance on late-train co-assessment patterns, not contamination from validation/test learners.

3.3 Graph Representation for Knowledge Tracing

We represent each fold’s knowledge structure as a directed graph $G_f = (\mathcal{C}, \mathcal{R}, \mathcal{E}_f)$ where nodes $c \in \mathcal{C}$ are knowledge components (hashed identifiers in the released pipeline), and edges carry relation type $r \in \mathcal{R} = \{\text{pre}, \text{sim}\}$ when that relation is instantiated on the corpus (prerequisite edges on all benchmarks; similarity edges only where the Q-matrix yields non-empty overlap). Prerequisite edges $(i, j) \in E_{\text{pre}}^f$ encode directed precedence inferred from temporal transitions; similarity edges $(i, j) \in E_{\text{sim}}^f$ encode co-assessment structure from the train-fold Q-matrix. Each edge stores provenance fields (fold id, train-only flag, source tag, support count, weight w_{ij}) exported as CSV for audit.

Table 2: Observed graph scale after DAG audit (fold 0) versus theoretical caps. Density is $|E_{\text{pre}}|/(|\mathcal{C}|(|\mathcal{C}| - 1))$ on prerequisite edges only. Even million-scale logs yield sparse, auditable G_f .

Dataset	$ \mathcal{C} $	$ E_{\text{pre}} $	Max out-deg.	Density	Cap regime
Junyi Academy	573	1,139	≤ 5	≈ 0.0035	$K=5000$
ASSIST'12	139	261	≤ 5	≈ 0.0136	$K=5000$
XES3G5M	455	701	≤ 5	≈ 0.0034	$K=5000$

Scalability limits.

Even when train folds contain tens of millions of interactions (Junyi: ≈ 25.9 M rows; Table 4), the exported graph remains bounded: nodes are at most the distinct knowledge component vocabulary $|\mathcal{C}|$; prerequisite edges are capped by a global top- K filter (default $K=5,000$) and a per-source out-degree cap (default $k=5$) before DAG pruning (Section 3.4). Transition counting scales linearly in train-fold sequence length; cycle detection operates on the sparse candidate set, not the full $|\mathcal{C}|^2$ adjacency. Table 2 reports observed node and edge counts after pruning on fold 0; density stays well below unity on all public benchmarks.

For graph-augmented KT baselines, G_f is materialised as a row-normalised adjacency matrix $A_f \in \mathbb{R}^{|\mathcal{C}| \times |\mathcal{C}|}$ (`pykt_graph_matrix.py`): prerequisite and similarity relations can be merged or consumed separately depending on the backbone (GKT uses a single KC graph; GIKT additionally uses question–KC bipartite structure from the train fold). Node features are not learned at graph-construction time; the protocol treats G_f as a *structural prior* consumed by downstream KT optimisers. This separation mirrors GNN practice on citation or knowledge graphs Kipf and Welling (2017); Hamilton et al. (2017) but with education-specific relation semantics and fold conditioning.

Technology stack: graph construction is NumPy/pandas/NetworkX; cycle detection uses Johnson’s algorithm Johnson (1975); KT training optionally delegates to pyKT Liu et al. (2022) with exported NPZ adjacency. All artefacts are versioned per dataset and fold under `data/graphs/<dataset>/fold_<f>/`.

3.4 Train-only Graph Construction

When $|\mathcal{F}_{\text{train}}^f|$ is very large, the protocol allows an optional *within-train* sampling step to bound edge-inference cost without opening a leakage channel (Figure 1). We draw $\tilde{\mathcal{I}}_{\text{tr}}^f \subseteq \mathcal{F}_{\text{train}}^f$ such that (i) all sampled interactions come from train learners only; (ii) per-knowledge component coverage is preserved by stratifying on KC frequency bins so rare knowledge components are not systematically dropped; and (iii) temporal order is preserved within each sampled learner sequence. All retained edges still carry fold provenance and train-only flags. *All reported experiments in this paper use the full train fold* (sampling disabled); Junyi (≈ 26 M train interactions) remains tractable under the top- K and per-source top- k filters below.

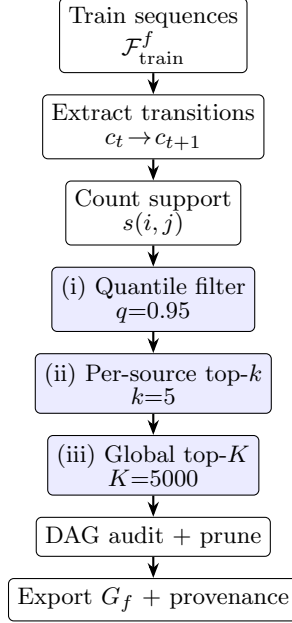


Fig. 4: Train-only prerequisite construction pipeline: temporal transitions are counted on $\mathcal{F}_{\text{train}}^f$ (or optional sample $\tilde{\mathcal{I}}_{\text{tr}}^f$), filtered, cycle-pruned, and exported with provenance fields (Section 3.5).

Prerequisite edges E_{pre} .

For uniformity across heterogeneous benchmarks, we infer E_{pre} from train-only temporal transitions on *all* three datasets, independently of whether an external prerequisite source is also available for that dataset. This deliberate choice decouples the audit pipeline from dataset-specific quality of expert annotations and ensures that every reported edge originates from a single, auditable inference rule. When an expert annotation is available for a benchmark (here, Junyi Academy), the protocol additionally performs ground-truth cross-validation as a separate audit step (Section 4.9).

Concretely, for each train-fold interaction sequence we extract consecutive KC transitions $c_t \rightarrow c_{t+1}$ with $c_t \neq c_{t+1}$, count their support $s(c_i, c_j)$, and retain candidate edges in three filtering stages: (i) a support-quantile filter that drops transitions below the q -th percentile (default $q = 0.95$); (ii) a per-source top- k cap (default $k = 5$) that retains, for each source KC, only the k destination knowledge components with highest support; and (iii) a global top- K cap on the number of retained candidate edges (default $K = 5,000$). Each retained edge carries the normalized support weight $w_{ij} = s(c_i, c_j)/s_{\text{max}}$ used by the cycle pruning step in Section 3.5. This candidate edge set is then passed unchanged to the DAG audit; cycle pruning operates only on the candidates returned by stages (i)–(iii).

Figure 4 makes the construction stages explicit; Figure 3(a) situates this block inside the wider data pipeline.

Similarity edges E_{sim} and theoretical properties.

We compute E_{sim} from a train-only Q-matrix using either Jaccard or PMI co-occurrence at the item level (default Jaccard with threshold $\tau=0.10$). Let $I(c) \subseteq \mathcal{Q}$ be the train-fold item set tagged with concept c , with $n = |\mathcal{Q}|$ items:

$$\text{sim}_J(c_i, c_j) = \frac{|I(c_i) \cap I(c_j)|}{|I(c_i) \cup I(c_j)|}, \quad (10)$$

$$\begin{aligned} \text{sim}_{PMI}(c_i, c_j) &= \log \frac{p(c_i, c_j)}{p(c_i)p(c_j)}, \\ p(c) &= \frac{|I(c)|}{n}, \\ p(c_i, c_j) &= \frac{|I(c_i) \cap I(c_j)|}{n}. \end{aligned} \quad (11)$$

Jaccard similarity is bounded in $[0, 1]$ and yields no E_{sim} when items are single-skill (ASSISTments 2012; Section 4.8). Full operator definitions and subgraph sampling details are in Supplementary Section S4. All graph artefacts are materialised per dataset and fold.

3.5 DAG Audit

For each fold-specific prerequisite edge set, we check whether E_{pre} is acyclic. If cycles are detected, we iteratively remove the lowest-confidence edge in a detected cycle and re-run the check until a topological ordering exists. Cycle detection uses a bounded simple-cycle enumeration procedure based on Johnson’s algorithm for enumerating elementary circuits in directed graphs Johnson (1975), capped at 100 representative cycles per detection round. We use this cap because the number of elementary cycles can grow rapidly with graph density; exhaustive enumeration is unnecessary for our audit, since the pruning loop only requires representative cycle witnesses. The detector cap does not affect loop correctness: the pruning loop terminates only when no further cycle can be detected, so the final retained graph is acyclic. Each removed edge is written to `results/reports/<dataset>_dag_pruning_log.csv` with its source, target, confidence score, and pruning reason. The audit reports node count, raw edge count, pruned edge count, retained edge count, root count, leaf count, and final topological-sort status. The claim is not that the retained DAG is the unique correct prerequisite hierarchy, but that the pruning rule and edge provenance are explicit and reproducible.

3.6 DAG Disruption Rate and Augmentation Operators

To quantify the structural effect of graph augmentations on prerequisite DAGs, we define the DAG Disruption Rate (DDR). Let A be an augmentation operator and $A(E_{pre}, p)$ the edge set obtained at perturbation strength p . Let $\text{pre}(A(E_{pre}, p)) \subseteq E_{pre}$ be the original prerequisite edges that remain with their original direction, and let $\text{rev}(A(E_{pre}, p))$ be the edges that appear with the opposite direction. We define

$$\text{DDR}(A, p) = \frac{|E_{pre} \setminus \text{pre}(A(E_{pre}, p))| + |\text{rev}(A(E_{pre}, p))|}{|E_{pre}|}. \quad (12)$$

We evaluate four label-agnostic families from GraphCL [You et al. \(2020\)](#) (`attr_mask`, `edge_drop`, random-walk `subgraph`, `node_drop`) plus one prerequisite-preserving operator (`prereq_preserve`) that removes only transitively redundant edges at the same per-edge budget as `edge_drop` (Section 4.6; formal definitions in Supplementary Section S4). Each operator is swept over $p \in \{0.05, 0.10, 0.20, 0.30\}$ with three random seeds on each of three train folds ($3 \times 3 = 9$ runs per setting); reported means aggregate across those runs. DDR is a structural diagnostic only: it measures how much prerequisite structure is removed or reversed by an augmentation, not whether a downstream KT model becomes better or worse.

3.7 Cold-start KC Stratification

We define cold-start at the KC level, distinct from user/item cold-start in recommender systems [Schein et al. \(2002\)](#). For fold f , let $n_f(c) = |\{(u, t, q, c, y) \in \mathcal{F}_{\text{train}}^f\}|$ be the number of train-fold interactions tagged with knowledge component c . Each KC is assigned to one of four frequency strata:

$$\begin{aligned}\mathcal{S}_1 &: n_f(c) < 20, \\ \mathcal{S}_2 &: 20 \leq n_f(c) < 100, \\ \mathcal{S}_3 &: 100 \leq n_f(c) < 500, \\ \mathcal{S}_4 &: n_f(c) \geq 500.\end{aligned}$$

Strata are assigned from train-fold counts only; test rows inherit their knowledge component stratum and aggregate AUC/ACC/NLL per stratum. This exposes tail-knowledge component behaviour that aggregate metrics hide and links rare-knowledge component edge support to $\text{ECR}_f^{\text{overlap}}$ (Table 1).

3.8 Ground-Truth Cross-Validation

When a benchmark provides an expert prerequisite annotation, the protocol performs an additional, optional audit step: comparing the train-only inferred E_{pre} against the expert edge set $E_{\text{pre}}^{\text{expert}}$. The procedure has four stages: (i) load the expert annotation and filter rows by a prerequisite-score threshold θ to obtain a directed expert edge set; (ii) align expert KC identifiers to the pipeline’s hashed `kc_id` space via a deterministic name-to-id mapping reconstructed from the preprocessing hash function; (iii) sweep top- K cutoffs over the inferred edge set and compute, at each K : edge precision/recall/F1, direction agreement, reachability F1 (over transitive closure), and node Jaccard; (iv) extract qualitative examples of disagreement in three categories: edges present only in inferred, edges present only in expert, and edges with opposite direction. The procedure is read-only with respect to the pipeline; no inferred edge is added or removed based on the comparison.

Table 3: Pros and cons of common KC-graph construction practices versus the leakage-controlled audit protocol (Section 3).

Practice	Advantages	Disadvantages	Leakage control
Full-log pooled graph (GKT/SKT default)	Simple; maximises transition counts	Held-out learners can support edges; ranking may inflate graph backbones	No
Train-only inference, no DAG audit	Fast; fold-clean edge evidence	Cycles may persist silently; augmentations unmeasured	Partial
Expert-only DAG (e.g. Junyi annotation)	Pedagogically curated structure	Unavailable on most benchmarks; not behavioural	N/A
End-to-end learned graph	Adaptive to data	Opaque provenance; hard to audit against splits	Unclear
Audit protocol (this paper)	Fold-specific provenance; bounded sparse G_f ; DDR + cold-start strata	Overhead; behavioural-expert edge F1 can be low	Yes

3.9 Comparison with common KC-graph practices

Table 3 contrasts how practitioners typically obtain KC graphs against the audit protocol developed here. The comparison is methodological, not a predictive leaderboard: our claim is that graph *provenance* should be a first-class experimental factor.

Adopt this protocol when (i) graph-augmented KT results inform deployment or publication comparisons, (ii) the concept graph is inferred from the same log used for evaluation, or (iii) reviewers or regulators require explicit leakage and cold-start reporting. Expert-only or full-log graphs may suffice for exploratory prototyping, but they should not be treated as audited benchmarks without provenance logs. When both expert and behavioural signals exist, ground-truth cross-validation (C4) supports treating them as complementary relations rather than substitutes (Section 4.9).

4 Experiments

4.1 Datasets

Benchmark roles.

XES3G5M Liu et al. (2023b) is our *primary* public benchmark for model-comparison claims: low consecutive KC-repeat ($\approx 21\%$, Supplementary Section S3), non-saturated headline AUC (≈ 0.875 for *simpleKT*), and the widest separation among graph backbones under train-only graphs. **ASSISTments 2012** is *secondary*: it stress-tests leakage ablation, cold-start strata, and DAG audit on a single-skill Q-matrix ($|E_{\text{sim}}|=0$). **Junyi Academy** Junyi Academy (2015) is a *saturated sanity* corpus for ground-truth cross-validation and dense E_{pre} topology; near-ceiling AUC (≈ 0.98 – 0.984) makes it unsuitable for headline graph-benefit claims. Synthetic C2/C5 are controlled tiny-graph sanity checks.

Table 4: Dataset statistics used in the P0 diagnostic protocol. Junyi Academy counts reflect the Chang et al. problem-level log (`junyi.ProblemLog_original.csv`) after preprocessing; #items and #KCs coincide because both map to the exercise column. Synthetic C2/C5 are companion sanity logs shipped with the preprocessing scripts.

Dataset	# learners	# items	# KCs	# interactions	Avg. seq. len.	Has DAG
Junyi Academy	247,606	722	722	25,925,992	104.7	Yes
ASSISTments 2012	29,018	53,086	265	2,711,602	93.4	No
XES3G5M	18,066	7,652	865	6,413,353	355.0	Yes
Synthetic C2	4,000	50	2	200,000	50.0	No
Synthetic C5	4,000	50	5	200,000	50.0	No

We instantiate the protocol on all five corpora. Junyi ships an expert-curated prerequisite annotation at the exercise-pair level. *In the main inference branch we do not consume this expert DAG:* E_{pre} for Junyi is built by the same train-only temporal inference rule used for ASSISTments and XES3G5M (Section 3.4). The expert DAG enters only the read-only ground-truth cross-validation sub-protocol (Section 4.9). ASSISTments provides a Q-matrix but no external prerequisite graph. XES3G5M provides hierarchical KC metadata parsed at the sequence level; E_{pre} is inferred by the shared train-only rule. Table 4 summarises the preprocessed datasets. The public benchmarks differ substantially in interaction volume, KC vocabulary size, and item-to-KC granularity. This heterogeneity is useful for auditing because the protocol must remain leakage-controlled under both curated and inferred graph settings. We use learner-based temporal splits with ratios (0.7, 0.1, 0.2) and seed 42; all edge provenance is regenerated for each run.

Benchmark repetitiveness.

Learner-disjoint splits remove cross-fold leakage but not within-log repetition shortcuts. Junyi and ASSISTments exhibit high consecutive KC-repeat rates (0.66–0.76), whereas XES3G5M repeats only 21% of the time (Supplementary Section S3). On the repetition-heavy corpora, *simpleKT* reaches $\text{AUC} \approx 0.968\text{--}0.984$ despite weak edge–outcome correlation (Table 5), whereas XES3G5M remains near 0.875 under the same protocol. This pattern inflates absolute deep-KT AUC where KC repetition is high, helps explain the near-null train-only versus full-log ablation (Section 4.5), and is why we anchor model-comparison claims on XES3G5M rather than on Junyi/ASSIST headline AUC.

4.2 Leakage and DAG Audit

Table 5 summarises the direct leakage metrics on the released runs (mean $\pm$ std over three folds). ECR^{flag} is identically zero on every dataset (fold means), confirming learner-disjoint splits and structural leakage freedom at the evaluation protocol level; $\text{ECR}^{\text{overlap}}$ stays near unity because recurrent transition patterns reappear among held-out learners even though edges are inferred strictly from train folds. The edge–outcome correlation is weak on all three public corpora ($|\rho| \approx 0.09\text{--}0.22$), so retained

Table 5: Direct leakage diagnostics per dataset (mean $\pm$ std over three folds). ECR_{flag} : learner-overlap indicator (Eq. 6). $\text{ECR}_{\text{overlap}}$: held-out pattern overlap (Eq. 7). EOC: exported monotone transform of edge-outcome Pearson $|\rho|$ (Eq. 8). TBMR: within-train temporal mixing (Eq. 9), not a train/test violation.

Dataset	ECR_{flag}	$\text{ECR}_{\text{overlap}}$	EOC	TBMR
ASSISTments 2012	0.000 ± 0.000	1.000 ± 0.000	1.423 ± 0.001	0.201 ± 0.001
Junyi Academy	0.000 ± 0.000	1.000 ± 0.000	1.449 ± 0.000	0.176 ± 0.001
Synthetic C2	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.733 ± 0.000
Synthetic C5	0.000 ± 0.000	1.000 ± 0.000	0.000 ± 0.000	0.133 ± 0.000
XES3G5M	0.000 ± 0.000	0.988 ± 0.001	1.420 ± 0.001	0.756 ± 0.001

Table 6: DAG audit summary (fold 0). *Raw* and *Final* are the prerequisite edge counts before and after lowest-confidence cycle pruning inside `dag_audit`; *Pruned* is the difference. The *Cycles* column reports representative cycles found before pruning (cap 100). All corpora pass topological sort with zero cycles after pruning on fold 0.

Dataset	$ V $	Raw $ E_{\text{pre}} $	Pruned	Final $ E_{\text{pre}} $	Roots	Leaves	Cycles (cap)
Junyi Academy	573	2,152	1,013	1,139	90	92	≥ 100
ASSIST'12	139	413	152	261	34	23	≥ 100
XES3G5M	455	1,162	461	701	129	73	≥ 100
Synthetic C2	2	2	1	1	1	1	1
Synthetic C5	3	2	0	2	1	1	0

edge mass is only weakly aligned with test-fold outcome summaries; ρ is undefined on degenerate synthetic graphs where edge-weight variance vanishes. TBMR is reported as the within-train temporal stress statistic of Section 3.2 (the share of edge evidence drawn from the late, post- τ_u tail of each learner’s train sequence); it ranges from 0.18 on Junyi to 0.75 on XES3G5M. Because folds are learner-disjoint ($\text{ECR}_{\text{flag}}=0$), these values quantify reliance on late train transitions rather than any cross-split contamination. We read $\text{ECR}_{\text{flag}}=0$ and provenance logs as **primary** pass/fail checks; overlap, ρ , and TBMR as **sanity/stress** indicators that can be near-degenerate under honest splits (Section 5.3). The leakage audit reports that all retained graph edges are train-only under the current fold configuration. Table 6 reports the DAG audit on fold 0 for all five corpora. The protocol produces non-empty acyclic prerequisite graphs after lowest-confidence cycle pruning on every dataset except the degenerate Synthetic C2 case ($|E_{\text{pre}}|=1$ after pruning).

On Junyi fold 0, 2,152 candidate edges prune to 1,139 acyclic edges; ASSISTments and XES3G5M retain 261 and 701 edges respectively after pruning (Table 6). Even train-only inference can induce cycles; the audit makes the pruning rule explicit rather than silently fixing the graph.

Figure 5 illustrates the *shape* of retained prerequisite structure on the densest public benchmark. These plots are not used for inference; they help reviewers compare graph density with DDR and baseline behaviour in later sections.

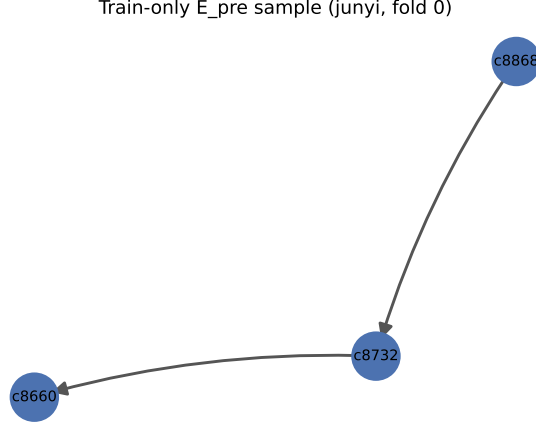


Fig. 5: Schematic visualisation of a high-support prerequisite chain in train-only E_{pre} on Junyi Academy after DAG audit (representative subgraph; the full graph contains hundreds to thousands of edges). ASSISTments 2012 and XES3G5M panels appear in Supplementary Figure S1.

4.3 Controlled test-fold edge injection

To verify that the audit metrics respond to graph-mediated leakage, we deliberately contaminate the XES3G5M fold-0 train-only prerequisite builder with 5% and 20% of test-fold interactions (Supplementary Table S17). Candidate-edge count ($|E_{\text{pre}}|$: 1162→1378→1417; counted *before* DAG pruning, whereas Table 6 reports the 701 edges retained *after* pruning on the clean fold) and $\text{TBM}R_f$ (0.754→0.776→0.779) increase monotonically with the injection rate, while $\text{ECR}_f^{\text{flag}}$ remains 0 because learner-id sets stay disjoint: flag-level checks alone cannot detect this contamination channel, which motivates the tiered audit. $\text{ECR}_f^{\text{overlap}}$ is already saturated at 1.0 on this transition-dense corpus (Section 3.1), so it carries no additional signal here. Retraining on the injected graphs (Supplementary Table S18) shows that headline AUC need not move in lockstep with the structural audit: GKT AUC rises from 0.810 (clean fold 0) to 0.860 at 20% injection, while *simpleKT*—which does not consume the graph—stays near 0.850–0.858. On this transition-dense corpus, injected test transitions disproportionately *densify* E_{pre} with high-support curriculum edges ($|E_{\text{pre}}|$ rises 1162→1417 before pruning; Table S17) that GKT’s graph convolution can exploit as additional relational context, whereas the sequence-only baseline is unaffected. The lift is therefore consistent with *graph-channel enrichment* rather than with improved split hygiene—and it illustrates why AUC alone is an unreliable leakage alarm: a contaminated graph can act as an accidental structural regulariser even when learner ids remain fold-disjoint ($\text{ECR}_f^{\text{flag}}=0$). Metric-level contamination is detectable in builder mass and $\text{TBM}R_f$ before practitioners would infer leakage from AUC alone, and tiered reporting remains necessary on real data where $\text{ECR}_f^{\text{overlap}}$ is already saturated at 1.0.

Table 7: Diagnostic baseline AUC summary (train-only graphs; three-fold learner CV means $\pm$ std). XES3G5M is the primary model-comparison column; Junyi is a saturated sanity corpus. SKT and DyGKT rows are fork wiring sanity checks only and are excluded from primary ranking claims. Per-dataset ACC/NLL and 95% bootstrap confidence intervals are in Supplementary Tables S1–S5; fold-level detail in Table 8.

Model	XES 3G5M	ASSIST 2012	Junyi	Synth C2	Synth C5
BKT	0.636 $\pm$ 0.002	0.669 $\pm$ 0.002	0.780 $\pm$ 0.001	0.835 $\pm$ 0.003	0.791 $\pm$ 0.002
AKT	0.875 $\pm$ 0.001	0.968 $\pm$ 0.001	0.984 $\pm$ 0.004	0.873 $\pm$ 0.003	0.822 $\pm$ 0.002
DKT	0.858 $\pm$ 0.000	0.963 $\pm$ 0.001	0.984 $\pm$ 0.004	0.862 $\pm$ 0.006	0.826 $\pm$ 0.001
<i>simpleKT</i>	0.875 $\pm$ 0.000	0.968 $\pm$ 0.000	0.984 $\pm$ 0.004	0.875 $\pm$ 0.004	0.829 $\pm$ 0.002
GKT	0.834 $\pm$ 0.001	0.961 $\pm$ 0.001	—	0.873 $\pm$ 0.005	0.810 $\pm$ 0.004
GIKT	0.878 $\pm$ 0.000	0.967 $\pm$ 0.000	0.982 $\pm$ 0.000	0.875 $\pm$ 0.005	0.820 $\pm$ 0.004
SKT	0.625 $\pm$ 0.001	0.953 $\pm$ 0.001	0.979 $\pm$ 0.006	0.708 $\pm$ 0.126	0.810 $\pm$ 0.003
DyGKT	0.752 $\pm$ 0.001	0.952 $\pm$ 0.001	0.978 $\pm$ 0.000	0.710 $\pm$ 0.136	0.826 $\pm$ 0.002
DGEKT	0.841 $\pm$ 0.000	0.960 $\pm$ 0.001	0.981 $\pm$ 0.000	0.710 $\pm$ 0.130	0.830 $\pm$ 0.002

4.4 Baseline Diagnostic

Table 7 sanity-checks preprocessing, learner splits, train-only graph export, and pyKT wiring across five corpora; it is not a SOTA claim. We read the table through the benchmark roles of Section 4.1. On the *primary* benchmark XES3G5M we report three **primary comparison backbones**: sequence-only *simpleKT*/AKT (≈ 0.875), stock pyKT GKT (≈ 0.834), and native GIKT (≈ 0.878). GKT trails *simpleKT* by ≈ 0.041 mean fold AUC on XES3G5M (95% CI $[-0.044, -0.038]$; Table S16). A GKT epoch-extended ablation (30 epochs, batch 32; *simpleKT* reference at 10 epochs, batch 64) narrows the gap to $\Delta \approx -0.038$ (95% CI $[-0.040, -0.035]$; Tables S21–S22): per split seed, mean $\Delta(\text{GKT} - \text{simpleKT})$ is ≈ -0.034 (seed 17), ≈ -0.040 (seed 42), and ≈ -0.041 (seed 1234). GIKT slightly exceeds the sequence leader (0.878 vs. 0.875; paired interval $[+0.003, +0.003]$, identical at three decimals on all three folds in Table 8—we report magnitudes rather than over-interpreting the degenerate interval width). SKT and DyGKT also appear in Tables 7 and 8 for fork wiring sanity only; they are excluded from primary ranking claims (Section 5.1). On ASSISTments (secondary), GKT trails by ≈ 0.007 AUC with near-ceiling scores for all leaders. On saturated Junyi, all deep models cluster near 0.98–0.984 and are poor discriminators of graph benefit. Per-dataset ACC/NLL and bootstrap intervals are in Supplementary Tables S1–S5.

Training parity and comparison scope.

Full epoch/batch parity is in Supplementary Table S15. Sequence-only checkpoints (AKT, DKT, *simpleKT*) use stock pyKT configurations (`lr`= 10^{-3} , `max_seq_len`=200, batch 64, 30 epochs). GKT is evaluated through the pyKT graph branch (batch 16, 10 epochs under the released budget; consumes exported $E_{\text{pre}}/E_{\text{sim}}$). Native GIKT (`src/models/gikt.py`) uses `emb_size`=64, `hidden_dim`=128, the same `lr`, batch 16, and 10 epochs. We do not claim bit-identical engineering parity across codebases; we align processed splits, data loaders, evaluation hooks, and early stopping (patience 5, best validation AUC). A one-step label-index misalignment in an early GIKT head

Table 8: Three-fold learner CV baseline summary (train-only graphs, valid+test positions). Mean $\pm$ std over folds; p_W : Wilcoxon signed-rank test vs. *simpleKT* (paired fold AUCs). SKT and DyGKT are fork wiring rows only (not primary ranking claims).

Dataset	Model	AUC	ACC	NLL	p_W
ASSIST '12	<i>simpleKT</i>	0.968 $\pm$ 0.000	0.916 $\pm$ 0.001	0.173 $\pm$ 0.001	—
	GKT	0.961 $\pm$ 0.001	0.907 $\pm$ 0.001	0.185 $\pm$ 0.001	0.250
	GIKT	0.967 $\pm$ 0.000	0.915 $\pm$ 0.001	0.175 $\pm$ 0.001	0.250
	SKT	0.953 $\pm$ 0.001	0.898 $\pm$ 0.001	0.198 $\pm$ 0.001	0.250
	DyGKT	0.952 $\pm$ 0.001	0.899 $\pm$ 0.001	0.200 $\pm$ 0.002	0.250
	DGEKT	0.960 $\pm$ 0.000	0.907 $\pm$ 0.001	0.186 $\pm$ 0.001	0.250
XES3G5M	<i>simpleKT</i>	0.875 $\pm$ 0.000	0.858 $\pm$ 0.001	0.324 $\pm$ 0.001	—
	GKT	0.834 $\pm$ 0.001	0.842 $\pm$ 0.000	0.364 $\pm$ 0.001	0.250
	GIKT	0.878 $\pm$ 0.000	0.860 $\pm$ 0.000	0.321 $\pm$ 0.001	0.250
	SKT	0.625 $\pm$ 0.001	0.800 $\pm$ 0.001	0.482 $\pm$ 0.001	0.250
	DyGKT	0.752 $\pm$ 0.001	0.813 $\pm$ 0.001	0.431 $\pm$ 0.001	0.250
	DGEKT	0.841 $\pm$ 0.000	0.844 $\pm$ 0.000	0.358 $\pm$ 0.001	0.250
Junyi Academy	<i>simpleKT</i>	0.984 $\pm$ 0.003	0.951 $\pm$ 0.008	0.121 $\pm$ 0.019	—
	GIKT	0.982 $\pm$ 0.000	0.946 $\pm$ 0.000	0.133 $\pm$ 0.001	1.000
	SKT	0.979 $\pm$ 0.005	0.938 $\pm$ 0.011	0.139 $\pm$ 0.023	0.250
	DyGKT	0.978 $\pm$ 0.000	0.934 $\pm$ 0.000	0.151 $\pm$ 0.001	0.250
	DGEKT	0.981 $\pm$ 0.000	0.943 $\pm$ 0.000	0.137 $\pm$ 0.001	0.250

was caught by fold-wise AUC inconsistency and corrected before Table 8 (Section 5.3). No family receives a hyperparameter search. Epoch caps differ in the primary release: sequence checkpoints train up to 30 epochs (batch 64) while graph backbones use 10-epoch budgets with smaller batches (Table S15), reflecting higher per-epoch cost. Tables S21–S22 report *GKT-only* epoch extensions with fold-aligned graph rebuilds per split seed (17, 42, 1234), pairing against *simpleKT* from Phase 3 trio reruns (10 epochs, batch 64): on seed 42, mean GKT AUC rises from 0.834 to 0.837 ($\approx +0.003$), narrowing the gap from ≈ -0.041 to $\Delta \approx -0.038$ (seed 42 CI $[-0.048, -0.027]$). Pooled over nine folds, $\Delta \approx -0.038$ (95% CI $[-0.040, -0.035]$). This ablation isolates the GKT epoch confound; it is *not* full compute parity (batch and reference-epoch asymmetry remain). The residual deficit remains an *observational* benchmarking boundary on XES3G5M under this protocol, not a causal claim that graph structure is uninformative. Relative to the literature default of pooling transition statistics before learner splitting in GKT/SKT pipelines Nakagawa et al. (2019); Tong et al. (2020), our train-only graphs isolate a provenance factor those papers do not audit.

Table 8 reports three-fold learner-CV means $\pm$ std and Wilcoxon p -values vs. *simpleKT*. Table 7 provides the same mean $\pm$ std format across datasets. With only three paired folds the Wilcoxon test is underpowered (minimum two-sided $p=0.25$); paired t -test p -values on the same fold AUCs appear in Supplementary Table S11. We treat fold-level Δ AUC magnitudes (Table 8) and Table S16 intervals as the primary inferential evidence for the XES3G5M primary trio on this protocol; exploratory ANOVA summaries appear only in Supplementary Tables S19–S20. The resulting *model-specific* ordering on XES3G5M within the primary trio—pyKT GKT as the under-performer relative to *simpleKT* across all three split seeds, native GIKT competitive or slightly above—is developed in Section 5.1.

Table 9: Train-only versus full-log graph ablation summary (fold 0). Columns report the largest absolute ΔAUC and ΔACC (full-log minus train-only) across GKT, GIKT, SKT, DyGKT, and DGEKT. Per-model train-only/full-log pairs are in Supplementary Tables S6–S10.

Dataset	max $ \Delta\text{AUC} $	max $ \Delta\text{ACC} $	Largest shift (model)
Junyi Academy	0.003	0.008	SKT ($\Delta\text{AUC}=-0.003$)
ASSISTments 2012	0.000	0.000	all models ≈ 0
XES3G5M	0.003	0.001	DyGKT ($\Delta\text{AUC}=-0.003$)
Synthetic C2	0.001	0.001	GKT/DGEKT
Synthetic C5	0.002	0.002	GKT ($\Delta\text{AUC}=+0.002$); GIKT ($\Delta\text{ACC}=-0.002$)

4.5 Graph Construction Ablation (Train-only vs. Full Log)

We contrast fold-specific train-only graph exports (`e\_pre\_train\_only.csv`, `e\_sim\_train\_only.csv`) against a *single* graph inferred from the entire preprocessed log (`data/processed/<dataset>/full\_log/`), holding learner splits and evaluation code fixed. Only the static graph artefact fed into the pyKT graph branch differs; KT optimisation still uses train-fold sequences only.

Table 9 summarises the largest train-only versus full-log shifts across GKT, GIKT, SKT, DyGKT, and DGEKT (fold 0). On the three public benchmarks, $|\Delta\text{AUC}|$ and $|\Delta\text{ACC}|$ stay at or below 0.003; Synthetic C5 shows the largest released shift ($|\Delta\text{AUC}|=0.002$ for GKT). Per-model pairs appear in Supplementary Tables S6–S10. Under our support quantile ($q=0.95$), top- k cap, and $K=5000$ filter, pooling learners before edge inference rarely changes surviving transitions on public benchmarks. Train-only construction is therefore *not* penalising headline metrics relative to this deliberate full-log variant on those corpora, without proving absence of leakage in general.

4.6 DAG Disruption Rate

Figure 6 and Table 10 show five consistent regimes across the three benchmarks. First, attribute masking leaves E_{pre} structurally unchanged and therefore gives $\text{DDR} = 0$ across perturbation strengths by construction. Second, random-walk subgraph sampling preserves connectivity inside the sampled subgraph and disrupts E_{pre} *sub-linearly* in p : because edges are removed only at the boundary of the visited subgraph, the disruption is consistently smaller than under edge dropping at the same p (e.g. on Junyi, $\text{DDR} = 0.16$ for subgraph vs. 0.20 for edge drop at $p = 0.20$). Third, edge dropping disrupts prerequisite structure approximately linearly in p , as expected when each edge is removed independently. Fourth, node dropping produces the largest disruption because removing a node also removes all incident prerequisite edges; at $p = 0.30$, node dropping disrupts roughly half of the prerequisite structure on all three benchmarks ($\text{DDR} \approx 0.52\text{--}0.54$).

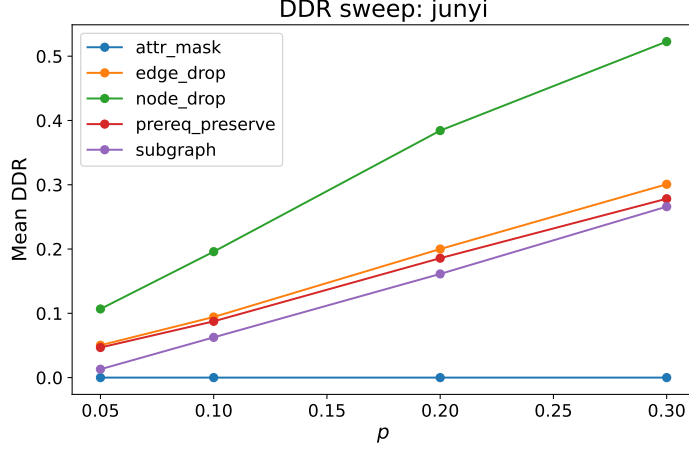


Fig. 6: DDR as a function of perturbation strength p on Junyi Academy for five graph augmentation families: four label-agnostic operators (`attr_mask`, `subgraph`, `edge_drop`, `node_drop`) and one prerequisite-preserving operator (`prereq_preserve`). Each marker plots the mean DDR averaged across nine stochastic runs (three random seeds on each of three train folds). The same ordering holds on ASSISTments 2012 and XES3G5M (Supplementary Figure S2): $\text{DDR}(\text{attr_mask}) = 0 < \text{DDR}(\text{subgraph}) \lesssim \text{DDR}(\text{prereq_preserve}) < \text{DDR}(\text{edge_drop}) \approx p < \text{DDR}(\text{node_drop})$.

DDR is an operator-level structural diagnostic, not a tautology.

A natural objection to the four label-agnostic families is that their DDR ordering merely re-states their definitions: an operator that deletes more edges disrupts more structure. To test whether DDR *discriminates* structure-aware from structure-agnostic augmentation rather than re-describing edge counts, we add a fifth, prerequisite-preserving operator (`prereq_preserve`). It draws the same per-edge budget p as `edge_drop`, but only removes edges that are *transitively redundant*—those whose endpoints remain connected through an alternative directed path—so that every reachability/precedence relation and the entire transitive-reduction backbone are preserved. Crucially, unlike `attr_mask`, this operator is *not* $\text{DDR}=0$ by construction: its disruption is an empirical quantity equal to p times the redundant-edge fraction of the graph. At every dataset and strength, `prereq_preserve` is strictly less disruptive than `edge_drop` at a matched budget (at $p = 0.30$, 0.278 vs. 0.301 on Junyi, 0.262 vs. 0.309 on ASSISTments, and 0.233 vs. 0.299 on XES3G5M). The size of this gap is *data-dependent*—a $\sim 8\%$ relative reduction on Junyi but $\sim 22\%$ on XES3G5M—and is determined by graph redundancy rather than by the operator definition. Because temporal-precedence inference generates many transitive shortcuts (if a precedes b precedes c , the edge $a \rightarrow c$ is also counted), the inferred graphs are highly redundant ($\approx 78\text{--}92\%$ of edges lie off the transitive reduction), which both explains the modest gap and shows that DDR responds to operator structure-awareness in a quantitative,

Table 10: Mean $\pm$ std DDR across three seeds and three train folds for the five augmentation families and four perturbation strengths (9 runs per cell). Subgraph sampling is random-walk based (Section 3.6); `prereq_preserve` removes only transitively redundant edges at the same per-edge budget as `edge_drop`. Lower is less disruptive.

Dataset	Family	$p=0.05$	$p=0.10$	$p=0.20$	$p=0.30$
Junyi Academy	<code>attr_mask</code>	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
	<code>subgraph</code>	0.013 ± 0.000	0.063 ± 0.009	0.161 ± 0.017	0.266 ± 0.021
	<code>prereq_preserve</code>	0.047 ± 0.002	0.088 ± 0.002	0.186 ± 0.004	0.278 ± 0.007
	<code>edge_drop</code>	0.050 ± 0.001	0.094 ± 0.002	0.200 ± 0.005	0.301 ± 0.008
	<code>node_drop</code>	0.107 ± 0.019	0.196 ± 0.026	0.384 ± 0.014	0.523 ± 0.010
ASSIST'12	<code>attr_mask</code>	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
	<code>subgraph</code>	0.012 ± 0.002	0.028 ± 0.003	0.071 ± 0.007	0.188 ± 0.023
	<code>prereq_preserve</code>	0.048 ± 0.010	0.087 ± 0.011	0.179 ± 0.014	0.262 ± 0.017
	<code>edge_drop</code>	0.055 ± 0.011	0.100 ± 0.013	0.212 ± 0.011	0.309 ± 0.009
	<code>node_drop</code>	0.092 ± 0.029	0.193 ± 0.027	0.389 ± 0.090	0.530 ± 0.078
XES3G5M	<code>attr_mask</code>	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000
	<code>subgraph</code>	0.022 ± 0.002	0.058 ± 0.005	0.145 ± 0.011	0.240 ± 0.010
	<code>prereq_preserve</code>	0.038 ± 0.005	0.072 ± 0.006	0.158 ± 0.011	0.233 ± 0.006
	<code>edge_drop</code>	0.049 ± 0.005	0.095 ± 0.005	0.204 ± 0.009	0.299 ± 0.002
	<code>node_drop</code>	0.110 ± 0.022	0.196 ± 0.027	0.386 ± 0.035	0.535 ± 0.027

non-tautological way. DDR can therefore serve as a concrete design target: an augmentation that respects prerequisite directionality is measurably rewarded with lower disruption at the same perturbation budget.

The relative ordering $\text{DDR}(\text{attr_mask}) < \text{DDR}(\text{subgraph}) \lesssim \text{DDR}(\text{prereq_preserve}) < \text{DDR}(\text{edge_drop}) < \text{DDR}(\text{node_drop})$ is consistent across all three benchmarks and all four perturbation strengths. We read these results as a structural diagnostic, not as a claim that any augmentation is beneficial or harmful for a downstream KT learner. They indicate that prerequisite-aware augmentation design is needed when graph augmentations are applied to educational DAGs: node-level operators are an order of magnitude more disruptive than attribute-level operators, with subgraph sampling and the prerequisite-preserving operator occupying an intermediate, structure-preserving regime.

4.7 DDR Downstream Stress Test

Table 11 connects DDR to an *exploratory* DGEKT-only retraining stress test on ASSISTments 2012 and XES3G5M (three folds; `edge_drop`, `node_drop`, `prereq_preserve`). Despite large structural disruption (`node_drop` mean DDR 0.493–0.555 at $p=0.30$), downstream AUC changes for *DGEKT* stay near zero and pooled DDR–AUC-drop correlations are weak ($r=0.13$ – 0.15 ; supplementary DDR-downstream figure). DDR therefore audits augmentation *structure*; it is not, on the evidence reported here, a universal proxy for accuracy degradation across graph-consuming backbones. Only DGEKT was retrained in this sweep; extending the same DDR→AUC protocol to GKT (and ideally GIKT) is the highest-priority follow-up for C2 practical value (Section 5.4).

Table 11: DDR vs. downstream graph-KT accuracy. For each dataset and model, the prerequisite graph E_{pre} is perturbed by each operator at strength p ; *AUC drop* is the mean decrease in test AUC relative to the unperturbed (DDR= 0) baseline graph across folds. Pooled correlations (DDR vs. AUC drop): assist2012/dgekt: $r=0.15$ ($p=4.7\text{e-}01$), $\rho=0.18$; xes3g5m/dgekt: $r=0.13$ ($p=5.1\text{e-}01$), $\rho=0.18$.

Dataset	Model	Operator	p	Mean DDR	Mean AUC	AUC drop
assist2012	dgekt	edge_drop	0.10	0.086	0.9600	0.0000
assist2012	dgekt	edge_drop	0.20	0.221	0.9601	-0.0000
assist2012	dgekt	edge_drop	0.30	0.304	0.9600	-0.0000
assist2012	dgekt	node_drop	0.10	0.171	0.9601	-0.0001
assist2012	dgekt	node_drop	0.20	0.369	0.9601	-0.0000
assist2012	dgekt	node_drop	0.30	0.493	0.9601	-0.0000
assist2012	dgekt	prereq_preserve	0.10	0.080	0.9601	-0.0001
assist2012	dgekt	prereq_preserve	0.20	0.206	0.9601	-0.0000
assist2012	dgekt	prereq_preserve	0.30	0.278	0.9600	0.0000
xes3g5m	dgekt	edge_drop	0.10	0.093	0.8417	0.0001
xes3g5m	dgekt	edge_drop	0.20	0.214	0.8416	0.0001
xes3g5m	dgekt	edge_drop	0.30	0.301	0.8417	0.0000
xes3g5m	dgekt	node_drop	0.10	0.177	0.8417	0.0001
xes3g5m	dgekt	node_drop	0.20	0.424	0.8414	0.0003
xes3g5m	dgekt	node_drop	0.30	0.555	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.10	0.071	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.20	0.168	0.8417	0.0001
xes3g5m	dgekt	prereq_preserve	0.30	0.233	0.8415	0.0003

4.8 Cold-start KC Diagnostic

Table 12 shows that aggregate AUC can hide per-stratum variation: on real benchmarks the best graph-aware family leads only in *very_cold* ($\Delta_{\text{max}}=0.002$ on ASSISTments, $+0.005$ on XES3G5M) and is tied or worse in *warm/hot* (mean $\Delta_{\text{max}}\approx-0.005$). Tiny very-cold counts ($n<200$ on Junyi/ASSIST) should not be over-interpreted. Synthetic C2/C5 route all test mass to *hot*; their positive Δ_{max} values reflect clean generator structure, not cold-start relief. GKT underperforms sequence leaders on synthetic hot rows, reinforcing model-specific graph benefit.

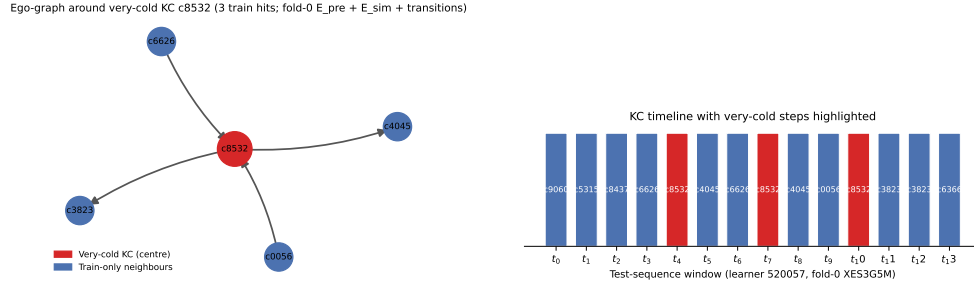
Figure 7 links Table 12 to concrete topology and sequence views. The ego-graph shows why rare knowledge components depend on neighbour structure; the timeline shows why aggregate AUC can obscure per-stratum behaviour when cold steps are sparse.

Similarity edges and dataset properties.

The similarity-edge rule of Section 3.4 is sensitive to the multi-skill density of the Q-matrix. On the train fold of ASSISTments 2012, 100% of items are tagged with exactly one KC, so $|I(c_i) \cap I(c_j)| = 0$ for every distinct concept pair, and both Jaccard and PMI yield no edges. On XES3G5M, 86.6% of train items are single-skill but the remaining multi-skill items generate 354 similarity edges at the default Jaccard threshold of 0.10. We treat this dataset-property dependency as a first-class outcome of the audit: a similarity rule appropriate for multi-skill benchmarks may need to be replaced or supplemented (e.g., by response-pattern correlation) on single-skill benchmarks. This is not a pipeline failure but a structural observation that protocol papers should report.

Table 12: Cold-start KC diagnostic summary. Column n counts fold 0 test interactions per stratum for *simpleKT*; *simpleKT* AUC and $\Delta_{\max}$ report three-fold means $\pm$ std ($\Delta_{\max}$: best graph-family AUC minus best sequence-only AUC). Per-model strata appear in Supplementary Tables S12–S13.

Dataset	Stratum	n	<i>simpleKT</i> AUC	$\Delta_{\max}$
Junyi Academy	very_cold	19	0.813 ± 0.324	$+0.000 \pm 0.000$
	cold	256	0.505 ± 0.239	$+0.000 \pm 0.000$
	warm	11,257	0.631 ± 0.014	-0.004 ± 0.006
	hot	$\approx 7.7 \times 10^6$	0.719 ± 0.000	-0.006 ± 0.000
ASSIST'12	very_cold	49	0.724 ± 0.100	-0.007 ± 0.012
	cold	707	0.772 ± 0.003	$+0.000 \pm 0.000$
	warm	9,017	0.756 ± 0.017	$+0.001 \pm 0.000$
	hot	$\approx 8.1 \times 10^5$	0.696 ± 0.002	-0.005 ± 0.000
XES3G5M	very_cold	161	0.817 ± 0.013	$+0.006 \pm 0.006$
	cold	2,592	0.804 ± 0.007	-0.000 ± 0.001
	warm	17,655	0.787 ± 0.003	-0.007 ± 0.000
	hot	$\approx 1.9 \times 10^6$	0.717 ± 0.000	-0.009 ± 0.000
Synth C2	hot	60,000	0.626 ± 0.002	$+0.008 \pm 0.000$
Synth C5	hot	60,000	0.630 ± 0.003	$+0.002 \pm 0.001$



(a) Cold-start KC ego-graph (fold 0 export). (b) Learner KC timeline with cold steps.

Fig. 7: Cold-start visualisation on XES3G5M (fold 0; very-cold stratum $n=161$ test interactions for *simpleKT* in Table 12). Panels are generated from released fold exports (`e_pre_train_only.csv`, `e_sim_train_only.csv`) and the pre-processed interaction log by `scripts/plot_kt_graph_figures.py`. (a) Ego-graph around a very-cold KC (red): neighbours combine E_{pre} , E_{sim} where present, and train-only temporal transitions. (b) A test-sequence window for one held-out learner with repeated very-cold steps; stratum labels derive from train-fold counts only (Section 3.7).

4.9 Ground-Truth Cross-Validation on Junyi

Junyi Academy [Junyi Academy \(2015\)](#) is the only benchmark in our study that ships with a publicly released expert prerequisite annotation (Chang et al.). The annotation enumerates directed exercise-pair relations with a continuous `Prerequisite_avg` score on a mean-opinion-score-like scale. We use $\theta = 0.5$ as the prerequisite filter, retaining 1,131 directed expert edges over 361 distinct exercise endpoints that participate in those edges. Following Section 3.8, expert exercise names are mapped to pipeline `kc_id` via the preprocessing hash function reconstructed from `data/raw/junyi/junyi_Exercise_table.csv`. All 1,131 expert edges align deterministically: the alignment audit reports `matched = 1,131`, `missingsrc = 0`, `missingdst = 0`, and alignment rate 1.0. We compare this expert edge set against the fold 0 train-only inferred list in `e\_pre\_train\_only.csv`, which contains 2,152 ranked directed candidates (the same file audited in Section 4.2; cycle pruning yields 1,139 retained acyclic edges).

Table 13: GT cross-validation: agreement between train-only inferred E_{pre} and expert prerequisite DAG on Junyi Academy ($|E| = 1131$ directed edges).

K (top inferred)	Edge prec.	Edge rec.	Dir. agr.	Rch. F1
200	0.235	0.042	0.346	0.008
500	0.178	0.079	0.225	0.021
800	0.181	0.128	0.280	0.062
$ E_{\text{expert}} = 1131$	0.183	0.183	0.261	0.104
1200	0.180	0.191	0.262	0.126
1800	0.153	0.243	0.278	0.260
2500	0.143	0.271	0.285	0.339
3500	0.143	0.271	0.285	0.339
5000 (cap)	0.143	0.271	0.285	0.339

Table 13 and Figure 8 show low edge-level overlap ($F1 \approx 0.18$ at $K=|\text{expert}|$) that plateaus as K grows, modest direction agreement (≈ 0.26 – 0.29 on shared pairs), but comparatively high node Jaccard (0.632 at $K=|\text{expert}|$; Supplementary Table S14 reports the full top- K sweep). Reachability F1 rises to 0.339 at large K , indicating recovery of indirect chains despite edge disagreement. Behavioural inference therefore encodes co-occurrence more than pedagogical ordering; it is not a low-cost substitute for expert curation and should be combined as a separate relation when both sources exist. Qualitative disagreement examples appear in Supplementary Section S5. The audit uses $\theta=0.5$ on a single seed and was not swept over stricter thresholds.

5 Discussion

5.1 Model-specific graph benefit under leakage-controlled evaluation

The baseline summary (Table 7) and fold-level diagnostics (Table 8) support a *bounded* benchmarking observation rather than a universal ranking reversal. We anchor the

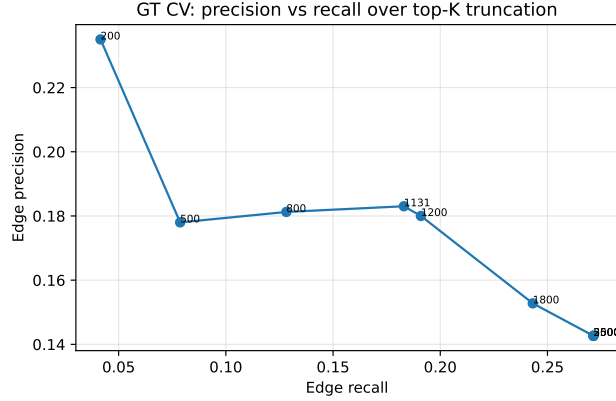


Fig. 8: Edge-level precision and recall of train-only inferred E_{pre} versus the expert prerequisite annotation on Junyi, across top- K cutoffs $K \in \{200, 500, 800, 1131, 1200, 1800, 2500, 3500, 5000\}$. The curve plateaus at $K \geq 2,500$ because the inferred edge pool tops out at 2,152 after DAG audit.

discussion on **XES3G5M** (primary; Section 4.1) because its lower KC-repeat and non-saturated AUC make backbone separation interpretable. Under leakage-controlled evaluation (train-only E_{pre} , optional train-only E_{sim} where populated, learner-based temporal split), explicit concept-graph benefit is *model-specific* within the **primary trio** (*simpleKT*, *pyKT* GKT, native GIKT) rather than uniform. On XES3G5M, *simpleKT* averages 0.875 AUC while GKT averages 0.834 under the released budget (mean $\Delta \approx -0.041$; 95% CI $[-0.044, -0.038]$, Table S16; fold stds in Table 8). A GKT epoch-extended rerun on split seed 42 reaches 0.837 AUC but still trails by pooled $\Delta \approx -0.038$ (95% CI $[-0.040, -0.035]$; Tables S21–S22; seed-level Δ stable at ≈ -0.034 to ≈ -0.041). GIKT (0.878) slightly *exceeds* the sequence leader (+0.003 mean AUC; interval $[+0.003, +0.003]$, tied at three decimals on all folds). Native SKT (0.625) and DyGKT (0.752) are reported as *fork wiring diagnostics* only: their large deficits may reflect under-tuned native stacks rather than inherent backbone limits and are not used to argue that graph-augmented KT fails. On ASSISTments (secondary), the same ordering direction holds but magnitudes shrink (GKT trails by ≈ 0.007 AUC with near-ceiling leaders). On saturated Junyi, all models cluster within ≈ 0.002 – 0.006 AUC of *simpleKT* and should not drive deployment conclusions. Crucially, train-only versus full-log ablation moves headline metrics by at most $|\Delta \text{AUC}| \leq 0.003$ on the three public benchmarks (Table 9), so the protocol’s primary *provenance* contribution is reproducibility and risk disclosure, not a demonstrated systematic flip of backbone rankings caused by leakage control alone.

We do not claim that graph-augmented KT is uninformative; the protocol removes a specific channel through which held-out information can reach the model—the concept graph itself. Three non-mutually-exclusive hypotheses are consistent with the observed ordering.

H1 – Leakage in prior evaluations.

A common practice in graph-augmented KT is to compute graph statistics from the full interaction log prior to train/test splitting. Under such practice, graph-augmented models can observe test-fold transitions through the graph during training. Our protocol enforces fold-specific edge inference and provenance logging. Table 9 instantiates the controlled comparison: identical learner splits and evaluation code, but graph statistics pooled over the full preprocessed log. Headline metrics on the three public benchmarks shift by at most $|\Delta\text{AUC}|\leq 0.003$; Synthetic C5 shows a modest shift ($|\Delta\text{AUC}|=0.002$). H1 remains relevant as a *risk hypothesis* about workflows that leak held-out mass into graph construction; it is not a causal claim that we have confirmed on these benchmarks. Table 9 caps how much headline leakage can explain the observed GKT deficit here ($|\Delta\text{AUC}|\leq 0.003$ on public corpora). Architectures that exploit cross-split pooling more aggressively might show larger shifts.

H2 – Aggregate-metric masking.

The cold-start diagnostic (Section 4.8) shows that per-stratum behaviour can differ from aggregate behaviour. Table 12 makes this concrete: on real data, the best graph-aware model is slightly better in the *very_cold* stratum, essentially tied in *cold*, and worse in *warm/hot*. On Synthetic C2/C5, where all test interactions are hot, GIKT/DGEKT improve over sequence-only baselines, but this reflects clean synthetic structure rather than cold-start relief.

H3 – Strong-backbone substitution.

Modern attention-based backbones such as AKT and *simpleKT* may already capture much relational information through long-range temporal attention. Additional structural signal at the concept-embedding level may then yield diminishing returns—a pattern weaker temporal backbones should benefit from more, all else equal.

These hypotheses are not exhaustive. Distinguishing among them requires re-running prior graph-KT papers under our protocol, which is beyond this paper’s scope. The narrower observation is that *under leakage-controlled evaluation on XES3G5M, backbone ordering within the primary trio can differ from near-ceiling aggregates on other corpora*, and benchmarking practice should treat graph provenance and training-budget parity as first-order reporting factors—while acknowledging that provenance control alone moved headline AUC by at most 0.003 on the public benchmarks studied here.

Why this strengthens the protocol.

If graph-augmented KT reliably outperformed sequence-only baselines regardless of graph provenance, leakage control would be secondary. The combination of (i) model-specific trio ordering on XES3G5M under train-only graphs, (ii) a near-null train-only/full-log ablation on public benchmarks, (iii) controlled injection showing scalar flags can miss graph-builder contamination (Tables S17–S18), and (iv) edge provenance plus GT cross-validation on Junyi, elevates graph auditing from optional hygiene to a deployment precondition. Combined with Section 4.9, which shows that inferred and

expert prerequisite signals are structurally distinct, graph quality cannot be assumed—it must be audited.

Caveats.

Baseline ordering uses three-fold means; DAG audit and GT cross-validation refer to fold 0 exports by default. GKT, AKT, DKT, and *simpleKT* are stock pyKT Liu et al. (2022) checkpoints; GIKT, SKT, DyGKT, and DGEKT are native implementations in our fork (`src/models/`). Epoch caps differ in the primary release (30 for sequence checkpoints vs. 10 for graph backbones; Table S15); Tables S21–S22 show GKT-only epoch extension recovers only $\approx +0.003$ AUC of the GKT deficit on seed 42, with pooled nine-fold $\Delta \approx -0.038$ under partially matched caps. The GKT–*simpleKT* gap on XES3G5M remains *observational* under fair budgets, not proof that graph signal is useless after exhaustive tuning. The severe SKT/DyGKT deficits in Tables 7 and 8 are flagged as likely implementation-budget artefacts and excluded from primary ranking claims. Stronger graph-KT architectures Tong et al. (2020); Cheng et al. (2024); Cui et al. (2024) may behave differently; re-auditing them under the same protocol is explicit follow-up (Section 5.4).

5.2 Broader implications

Scalability and broader domains.

The protocol is dataset-agnostic: cost scales with train-fold transition counting and capped cycle enumeration, not KT training. Sparsity caps keep audit tractable on million-scale logs Choi et al. (2020). The similarity rule may need domain-specific co-occurrence when Q-matrices are single-skill; GT cross-validation generalises wherever expert DAGs exist; DDR applies to any directed curriculum graph consumed by KT or pretraining pipelines.

Implications for graph-enhanced KT.

The protocol shows that KC graph construction should be treated as a first-class experimental component rather than an unexamined preprocessing step. Fold-specific edge inference, provenance logging, and DAG validation make it possible to identify leakage risks and graph construction choices that would otherwise remain hidden. The DDR results further show that generic graph augmentations can substantially disturb prerequisite structure, especially when they operate at the node level, whereas a prerequisite-preserving operator that protects the transitive-reduction backbone is measurably less disruptive at the same perturbation budget. The exploratory DGEKT-only downstream stress test in Section 4.7 adds an important qualification: on *this* backbone and on ASSISTments/XES3G5M only, large structural disruption did not produce a measurable AUC drop in our retraining sweep. We do not extrapolate that null predictive sensitivity to GIKT, GKT, SKT, DyGKT, or other graph consumers. Future augmentation papers should report both structural damage (DDR) and backbone-specific downstream retraining evidence.

Cold-start reporting.

The cold-start diagnostic shows why KT studies should not rely only on aggregate AUC. Rare knowledge components may behave differently from frequent knowledge components, and apparent gains or high scores on small strata may reflect class imbalance rather than genuine modelling improvement. Reporting per-stratum metrics gives reviewers a clearer view of whether a method is robust across the KC frequency distribution. In our results, graph-aware models show their narrowest real-data advantage in the very-cold stratum, while synthetic hot-stratum gains reflect clean concept structure rather than cold-start relief. Figure 5 links this diagnostic to concrete graph topology on the densest benchmark; Figure 7 plots a fold 0 XES3G5M ego-graph and learner timeline from released exports; sparser E_{pre} on ASSISTments and XES3G5M (Supplementary Figure S1) still exhibit cold strata. Very-cold counts remain small ($n=19\text{--}161$ per dataset in Table 12); stratum-level AUC therefore carries high variance and should be read alongside n and fold-level dispersion.

Deployment decision vignettes.

The audit is most actionable when an output maps to a concrete deployment choice:

- **Similarity-edge feasibility (ASSISTments).** Jaccard similarity yields $|E_{\text{sim}}|=0$ under the single-skill Q-matrix (Section 4.8). A practitioner should not deploy a graph model that *requires* similarity edges under this rule without changing the co-occurrence definition or supplying an external relation.
- **Provenance versus headline gain (all public benchmarks).** Train-only versus full-log ablation shifts AUC by at most 0.003 (Table 9). The actionable conclusion is to enforce train-only graph exports for reproducibility and cross-paper comparability, not to expect large accuracy corrections on these corpora.
- **Metric tiering under controlled injection (XES3G5M).** Deliberate test-fold injection increases graph-builder mass while $\text{ECR}_f^{\text{flag}}=0$ (Tables S17–S18). Practitioners should treat fold-disjoint splits as necessary but insufficient and retain provenance logs plus builder-level stress tests before trusting scalar leakage summaries alone.

These vignettes complement—rather than replace—backbone comparisons: C5 reports model-specific trio ordering on XES3G5M under fixed budgets, with Table S21–S22 showing that GKT epoch extension recovers only $\approx +0.003$ AUC of the GKT deficit on seed 42 (pooled $\Delta \approx -0.038$ under partially matched caps).

Behavioural versus expert prerequisite curation.

The ground-truth cross-validation on Junyi (Section 4.9) shows that train-only behavioural inference and expert prerequisite annotation are correlated at the node level but structurally distinct at the edge level. Two implications follow. First, behavioural inference cannot be treated as a low-cost approximation of expert curation, and papers that report graph-augmented KT results on behaviourally-inferred graphs should not claim those results transfer directly to settings with expert prerequisite graphs (or vice versa). Second, the two signals are complementary: in the most informative setup, a graph-augmented KT model would consume both an inferred

behavioural graph and an expert prerequisite graph as separate relations, rather than treating either as a substitute for the other. We do not test this combined setting in the present paper, but the cross-validation provides the quantitative basis for proposing it.

Implications for graph-KT benchmarking.

The bounded observation in Section 5.1 has direct implications for how the graph-KT subfield reports gains. Benchmark tables that omit graph-construction provenance, cold-start strata, and training-budget parity are difficult to compare across papers even when aggregate AUC is similar. We do not advocate dismissing reported gains in the graph-KT literature; we advocate *disaggregating* them. A reported AUC improvement of Δ on a benchmark should ideally be decomposed into three components: the part that survives leakage control, the part that depends on graph-construction choices (relation type, threshold, edge cap, encoder), and the part that depends on backbone strength. The protocol in this paper supplies the first component; the cold-start diagnostic supplies a partial view of the second; the third remains an open methodological question.

A practical recommendation follows. When introducing a new graph-augmented KT model, papers should report at minimum: (a) the fold-specific provenance of every retained graph edge; (b) the cold-start stratum on which the gain is most concentrated; (c) the sequence-only baseline matched in compute and hyperparameter budget; (d) when an expert prerequisite annotation is available, the ground-truth cross-validation of the consumed graph against that annotation, using the edge / direction / reachability / node-Jaccard metric family used in Section 4.9; and (e) whether accuracy is actually sensitive to structure-preserving versus structure-damaging graph perturbations. Without these reports, an aggregate-AUC improvement is difficult to attribute. The protocol presented here makes these reports tractable.

5.3 Threats to Validity and Metric Interpretation

Three artefacts invite misreading: the near-null full-log ablation (Table 9), aggressive cycle pruning (Section 4.2), and scalar leakage metrics that are near-degenerate under learner-disjoint splits (Table 5).

The ablation null shift does *not* negate leakage control: graph statistics must not pool held-out learners regardless of backbone sensitivity. Under our filters, pooling rarely changes surviving transitions on public benchmarks; synthetic logs remain more sensitive. We therefore tier audit outputs (Section 3.1): **primary**—provenance logs, $\text{ECR}_f^{\text{flag}}=0$, and GT cross-validation when available; **empirical bounds**—full-log ablation Δ and cold-start strata; **sanity/stress**— $\text{ECR}_f^{\text{overlap}}$, $|\rho_f|$, and TBM_f . Near-unity overlap and $\text{ECR}_f^{\text{flag}}=0$ are expected under honest learner-disjoint construction and do not replace provenance or GT audits.

Cycle pruning resolves conflicts by dropping the lowest-weight edge per detected cycle (Johnson enumeration, capped at 100 witnesses per round); pruned mass reflects weak or bidirectional transitions rather than arbitrary deletion.

TBM_f is a within-train temporal stress statistic at a 70% chronological boundary inside each learner’s train fold—not cross-split leakage ($\text{ECR}_f^{\text{flag}}=0$). Dense multi-skill

co-assessment on XES3G5M and toy-graph structure on Synthetic C2 explain the largest values.

The audit also caught a one-step label misalignment in an early GIKT head (`src/models/gikt.py`), flagged by inconsistent fold AUCs and corrected before reporting Table 8.

Limitations.

Three-fold CV limits inferential power of fold-level tests ($p_{\min}=0.25$ for Wilcoxon). Primary XES3G5M ranking claims rely on fold-level Δ AUC magnitudes (Table 8) and Table S16 intervals (`scripts/bootstrap_auc_ci.py`; paired- t over three folds by default, matching fold-level Δ AUC; optional `--learner-bootstrap` on exported parquets). The GKT-*simpleKT* gap is *observational* under fixed epoch budgets (30 vs. 10; Table S15); Table S21–S22 shows the deficit shrinks only slightly ($\approx +0.003$ AUC on seed 42; pooled $\Delta \approx -0.038$) when GKT is trained with extended epoch caps, so it is not fully explained by the released 10-vs-30 epoch asymmetry alone. Tables S21–S22 pair GKT epoch-extended runs (30 epochs, batch 32) against *simpleKT* from Phase 3 trio reruns (10 epochs, batch 64), so training caps remain only partially matched; a 30-epoch *simpleKT* reference on the same nine folds is required before stronger matched-budget claims. Multi-seed GKT replication requires sequential fold-aligned graph rebuilds per split seed because default processed exports omit a seed suffix in the path layout. Native SKT/DyGKT runs are wiring diagnostics, not tuned-backbone benchmarks. Controlled injection (Section 4.3, Tables S17–S18) demonstrates metric-level sensitivity. ASSISTments yields no E_{sim} under item-overlap Jaccard; GT cross-validation is Junyi-only at $\theta=0.5$; pyKT GKT on Junyi Academy is omitted because dense multi-million interaction graphs make the stock GKT wrapper impractical under our compute budget (Tables S1, 7); downstream DDR retraining covers DGEKT only.

5.4 Future work

Three extensions follow naturally from the audit boundary established here. **Continuous and online graph updating.** Deployed tutoring systems accumulate new interactions over time; G_f should be refreshable from post-deployment logs under explicit temporal cutoffs so that graph updates do not re-pool held-out or future learners. An online variant would extend the tiered leakage diagnostics (ECR, TBMR) to sliding windows and report when structural drift warrants retraining versus graph-only patches. **Feedback-aware graph construction.** Behavioural prerequisite inference currently weights co-occurrence and temporal precedence. Future work can condition edge retention on learning gain, hint usage, or remediation outcomes while keeping the train-only provenance contract; the edge-outcome diagnostic $|\rho|$ (Eq. (8)) provides a starting point for detecting target-conditional artefacts. **Broader backbone and operator coverage.** Re-auditing additional graph consumers under the same protocol, and testing hierarchy-respecting augmentations beyond DDR, would clarify whether structural diagnostics transfer across encoders. The three-seed GKT replication in Tables S21–S22 establishes a reproducible multi-seed template; extending it to additional corpora and to full compute parity (including a 30-epoch *simpleKT* reference at batch 64 on the same folds) remains open. **DDR downstream coverage.** The

structural operator sweep is backbone-complete at the graph level, but downstream retraining was run only for DGEKT under compute budget. A minimal next step is a matched GKT retraining sweep on ASSISTments and XES3G5M so C2 reports at least two graph consumers before claiming (or refuting) predictive sensitivity to DDR.

6 Conclusion

We presented a leakage-controlled audit protocol for constructing and evaluating concept graphs in graph-enhanced KT. The contribution targets *Applied Intelligence* as a deployment and reproducibility layer: a practitioner checklist, provenance artefacts, tiered leakage diagnostics, DDR augmentation metrics, ground-truth cross-validation where expert graphs exist, and cold-start KC stratification. On XES3G5M (primary), ASSISTments (secondary), and Junyi (saturated sanity), the audit produces acyclic train-only prerequisite graphs after transparent cycle pruning. The DDR sweep separates augmentation families into consistent structural regimes across all three public corpora. The prerequisite-preserving operator is strictly less disruptive than edge dropping at a matched budget, confirming that DDR separates structure-aware from structure-agnostic operators at the graph level. An exploratory DGEKT-only downstream stress test on ASSISTments and XES3G5M shows that this structural ordering does not automatically imply predictive sensitivity for *that* backbone ($r=0.13\text{--}0.15$; mean AUC drops near zero).

Under fixed training budgets on XES3G5M, the primary trio shows a *model-specific* pattern: GKT trails *simpleKT* by ≈ 0.041 mean AUC (95% CI $[-0.044, -0.038]$; Table S16) while GIKT remains competitive; GKT epoch-extended training (Tables S21–S22; *simpleKT* reference unchanged) recovers only $\approx +0.003$ AUC on seed 42, leaving pooled $\Delta \approx -0.038$ (95% CI $[-0.040, -0.035]$). Train-only versus full-log ablation moves headline AUC by at most 0.003. Ground-truth cross-validation on Junyi shows high node overlap but weak edge agreement. Cold-start strata reveal a narrow graph-aware advantage only in very-cold knowledge components on real data, with small n and high variance. Practitioners can apply the five-item checklist in Section 5.2 before reporting new graph-augmented KT results. Section 5.4 outlines continuous graph updating and feedback-aware construction as next steps. Reproduction artefacts are at https://github.com/tuanymc/p0_project.git.

Acknowledgements. The authors thank the maintainers of the Junyi Academy, ASSISTments, and XES3G5M datasets and of the pyKT benchmark library, whose public releases made this study possible.

Declarations

Funding. The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval. This study uses only publicly available, de-identified knowledge-tracing benchmark datasets under their respective licences. No human-subjects experiments were conducted and no ethics approval was required. No claims about real-world learning outcomes are made.

Consent to participate / Consent for publication. Not applicable; the study involves no identifiable individuals.

Data availability. All datasets analysed in this study are publicly available: Junyi Academy (PSLC DataShop, <https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1198>), ASSISTments 2012, and XES3G5M. Preprocessing scripts, fold-specific graph exports (`e\_pre\_train\_only.csv`, audit logs), and all derived tables and figures shipped with the repository are produced by the documented pipeline; raw interaction files are downloaded separately under each dataset licence. Reproducers regenerate graphs with `python-msrc.graph_builder` after preprocessing.

Code availability. Source code, configurations, pipeline scripts, and result artefacts are available at https://github.com/tuanymc/p0_project.git. Paper-facing tables with fold-level mean $\pm$ std and inferential summaries are regenerated by `scripts/generate_phase_c_tables.py` (invoked from `scripts/generate_paper_artifacts.py`); Δ AUC intervals (Table S16) use `scripts/bootstrap_auc_ci.py` (paired- t over three folds by default; `--learner-bootstrap` for optional row-level resampling on parquets). Supplementary tables and figures are provided in the separate supplementary material document.

Supplementary material. The supplementary material document is provided alongside this manuscript and is indexed at its opening page. Tables S1–S5: per-dataset baseline diagnostics; S6–S10: train-only versus full-log ablation; S11: paired significance tests on public benchmarks; S12–S13: cold-start strata; S14: ground-truth metrics including node Jaccard; S15: training parity; S16: Δ AUC intervals for primary pairs; Tables S17–S18 report controlled leak-injection metrics and downstream AUC on injected graphs. Tables S21–S22 report the GKT epoch-extended ablation (seed 42 and multi-seed replication). Tables S19–S20 report exploratory ANOVA summaries (Section 16 in the supplementary material). Section S3: sequence-autocorrelation statistics; Sections S4–S5: augmentation-operator definitions and ground-truth disagreement examples. Figures S1–S2: multi-benchmark train-only E_{pre} samples and DDR curves; Figure S3: DDR downstream scatter plot. Cold-start panels in the main manuscript (Figure 7) are generated from fold 0 exports.

Author contributions. D. M. Tuan conceived the protocol, implemented the leakage-controlled pipeline, ran the experiments, and drafted the manuscript. N. K. Trinh and N. T. Duong contributed to the experimental implementation, data preprocessing, and result analysis. N. Q. Khanh contributed to manuscript revision and technical review. N. V. Hau and L. H. Son supervised the work, advised on the experimental design, and revised the manuscript. All authors read and approved the final manuscript.

References

- Abdelrahman G, Wang Q, Nunes BP (2023) Knowledge tracing: A survey. *ACM Computing Surveys* 55(11):1–37. <https://doi.org/10.1145/3569576>
- Badran Y, Preisach C (2024) Addressing label leakage in knowledge tracing models. *arXiv preprint arXiv:240315304* <https://doi.org/10.48550/arXiv.2403.15304>
- Bai Y, Liu Z, Guo T, et al (2025) Prerequisite relation learning: A survey and outlook. *ACM Computing Surveys* 57. <https://doi.org/10.1145/3733593>
- Chang HS, Hsu HJ, Chen KT (2015) Modeling exercise relationships in e-learning: A unified approach. In: *Proceedings of the 8th International Conference on Educational Data Mining (EDM)*, pp 532–535, URL <http://www.educationaldatamining.org/EDM2015/proceedings/short532-535.pdf>
- Cheng K, Peng L, Wang P, et al (2024) DyGKT: Dynamic graph learning for knowledge tracing. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp 377–388, <https://doi.org/10.1145/3637528.3671773>
- Choi Y, Lee Y, Shin D, et al (2020) EdNet: A large-scale hierarchical dataset in education. In: *International Conference on Artificial Intelligence in Education (AIED)*, pp 69–73, https://doi.org/10.1007/978-3-030-52240-7_13
- Corbett AT, Anderson JR (1994) Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction* 4(4):253–278. <https://doi.org/10.1007/BF01099821>
- Cui C, Yao Y, Zhang C, et al (2024) DGEKT: A dual graph ensemble learning method for knowledge tracing. *ACM Transactions on Information Systems* 42(3):1–24. <https://doi.org/10.1145/3638350>
- Ghosh A, Heffernan NT, Lan AS (2020) Context-aware attentive knowledge tracing. In: *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp 2330–2339, <https://doi.org/10.1145/3394486.3403282>
- Hamilton WL, Ying R, Leskovec J (2017) Inductive representation learning on large graphs. In: *Advances in Neural Information Processing Systems (NeurIPS)*, pp 1024–1034, <https://doi.org/10.48550/arXiv.1706.02216>
- Hassani K, Khasahmadi AH (2020) Contrastive multi-view representation learning on graphs. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp 4116–4126, <https://doi.org/10.48550/arXiv.2006.05582>
- Heffernan NT, Heffernan CL (2014) The ASSISTments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*

- 24(4):470–497. <https://doi.org/10.1007/s40593-014-0024-x>
- Johnson DB (1975) Finding all the elementary circuits of a directed graph. *SIAM Journal on Computing* 4(1):77–84. <https://doi.org/10.1137/0204007>
- Junyi Academy (2015) Junyi academy online learning activity dataset. PSLC DataShop, URL <https://pslcdatashop.web.cmu.edu/DatasetInfo?datasetId=1198>, accessed 2026-06-08
- Kapoor S, Narayanan A (2023) Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4(9). <https://doi.org/10.1016/j.patter.2023.100804>
- Kaufman S, Rosset S, Perlich C, et al (2012) Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6(4):1–21. <https://doi.org/10.1145/2382577.2382579>
- Kipf TN, Welling M (2017) Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.1609.02907>
- Lee W, Chun J, Lee Y, et al (2022) Contrastive learning for knowledge tracing. In: Proceedings of the ACM Web Conference (WWW), pp 2330–2338, <https://doi.org/10.1145/3485447.3512105>
- Liu Z, Liu Q, Chen J, et al (2022) pyKT: A Python library to benchmark deep learning based knowledge tracing models. In: Advances in Neural Information Processing Systems (NeurIPS), pp 13248–13261, <https://doi.org/10.52202/068431-1347>
- Liu Z, Liu Q, Chen J, et al (2023a) simpleKT: A simple but tough-to-beat baseline for knowledge tracing. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.2302.06881>
- Liu Z, Liu Q, Guo T, et al (2023b) XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. In: Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track, pp 40643–40662, <https://doi.org/10.52202/075280-1429>
- Nakagawa H, Iwasawa Y, Matsuo Y (2019) Graph-based knowledge tracing: Modeling student proficiency using graph neural network. In: IEEE/WIC/ACM International Conference on Web Intelligence (WI), pp 156–163, <https://doi.org/10.1145/3350546.3352513>
- Pan L, Li C, Li J, et al (2017) Prerequisite relation learning for concepts in MOOCs. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL), pp 1447–1456, <https://doi.org/10.18653/v1/P17-1133>

- Pandey S, Karypis G (2019) A self-attentive model for knowledge tracing. In: Proceedings of the 12th International Conference on Educational Data Mining (EDM), pp 384–389, URL https://drive.google.com/file/d/18d_X6AXkPMhiHFQ2POarstVbX_7oMdFM
- Piech C, Bassen J, Huang J, et al (2015) Deep knowledge tracing. In: Advances in Neural Information Processing Systems (NeurIPS), pp 505–513, <https://doi.org/10.48550/arXiv.1506.05908>
- Schein AI, Popescul A, Ungar LH, et al (2002) Methods and metrics for cold-start recommendations. In: Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 253–260, <https://doi.org/10.1145/564376.564421>
- Shen S, Liu Q, Chen E, et al (2020) Convolutional knowledge tracing: Modeling individualization in student learning process. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 1857–1860, <https://doi.org/10.1145/3397271.3401288>
- Thakoor S, Tallec C, Azar MG, et al (2022) Large-scale representation learning on graphs via bootstrapping. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.2102.06514>
- Tong H, Wang Z, Liu Q, et al (2022) HGKT: Introducing problem schema with hierarchical exercise graph for knowledge tracing. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, pp 405–415, <https://doi.org/10.1145/3477495.3532004>
- Tong S, Liu Q, Huang W, et al (2020) Structure-based knowledge tracing: An influence propagation view. In: IEEE International Conference on Data Mining (ICDM), pp 541–550, <https://doi.org/10.1109/ICDM50108.2020.00063>
- Veličković P, Fedus W, Hamilton WL, et al (2019) Deep graph infomax. In: International Conference on Learning Representations (ICLR), <https://doi.org/10.48550/arXiv.1809.10341>
- Xia J, Wu L, Chen J, et al (2022) SimGRACE: A simple framework for graph contrastive learning without data augmentation. In: Proceedings of the ACM Web Conference (WWW), pp 1070–1079, <https://doi.org/10.1145/3485447.3512156>
- Yang Y, Shen J, Qu Y, et al (2021) GIKT: A graph-based interaction model for knowledge tracing. In: Machine Learning and Knowledge Discovery in Databases (ECML-PKDD 2020), pp 299–315, https://doi.org/10.1007/978-3-030-67658-2_18
- You Y, Chen T, Sui Y, et al (2020) Graph contrastive learning with augmentations. In: Advances in Neural Information Processing Systems (NeurIPS), pp 5812–5823, <https://doi.org/10.48550/arXiv.2010.13902>

- Zhang J, Shi X, King I, et al (2017) Dynamic key-value memory networks for knowledge tracing. In: Proceedings of the 26th International Conference on World Wide Web (WWW), pp 765–774, <https://doi.org/10.1145/3038912.3052580>
- Zhu Y, Xu Y, Yu F, et al (2021) Graph contrastive learning with adaptive augmentation. In: Proceedings of the Web Conference (WWW), pp 2069–2080, <https://doi.org/10.1145/3442381.3449802>